Vibe Coding: Practice, Performance, Productivity, and Risk—A State-of-the-Art Review

Dominik L. Michels
Computational Sciences Group, KAUST

Mutaz Abu Ghazaleh
MAG Tech AI

François Lazzari
GN TEQ

Nabil Kassem
GN TEQ

Jonathan Klein
Computational Sciences Group, KAUST

arXiv:2608.20446v1 [cs.SE] 20 Aug 2026

Abstract

Vibe coding — AI-assisted software development in which the developer describes intent in natural language and validates results by running rather than reading the generated code — was named by Andrej Karpathy in February 2025 and produced its first body of empirical evidence within seventeen months. This state-of-the-art review assembles that evidence across a cross-disciplinary corpus spanning software engineering, human–computer interaction, labour economics, security research, governance, and education. We survey the model landscape, the tool ecosystem, and the performance record by task type, finding the early benchmarks saturated but task-level capability uneven: reliable code generation alongside weak fault detection and hard-to-audit documentation. The productivity record is at first contradictory: peer-reviewed field experiments report +26% more tasks per week, independent randomised trials measure a 19% slowdown, and team-level telemetry shows code-review time up +441%. We argue these readings are consistent once measurement method, scope, and time horizon are held constant, and identify six patterns behind the dispersion, among them effect-shrinkage under broader measurement, self-report diverging from independent measurement, output volume conflated with productivity, and bold claims walked back once tested over longer horizons. We further document security failures in deployed applications, code-quality degradation visible in large-scale code and developer telemetry, unsettled copyright exposure, and evidence of skill atrophy. The review closes with the open research questions and one falsifiable conjecture: that the gains are real on new code and shrink or reverse on mature codebases, which would account for most of the disagreement in the record.

1 Introduction

1.1 Background

Artificial intelligence (AI) research began formally with Alan Turing's 1950 test for machine intelligence [85] and passed through several distinct phases before arriving at the current era of large language models (LLMs). The symbolic AI and expert-systems wave of the 1960s–1980s was followed by a machine-learning (ML) resurgence built on statistical methods (such as support vector machines, kernel methods, or random forests) and a deep-learning breakthrough catalysed by *AlexNet*'s ImageNet victory in 2012 [59]. The transformer architecture introduced by Vaswani and colleagues in 2017 [109] shifted the frontier from task-specific vision and speech models to general-purpose sequence models, and the scaling of transformer pre-training on internet text produced the *GPT-3* language model in 2020 and, following its instruction-tuned variant, the *ChatGPT* service in November 2022 that brought AI-generated text into mainstream use.

Within the broader arc of AI, generative models constitute a distinct lineage that runs from generative adversarial networks in 2014 through variational autoencoders, diffusion models and the full *GPT* family (*GPT-1* through *GPT-4*) to the image generators *DALL-E*, *Stable Diffusion* and *Midjourney*, the multimodal systems *GPT-4o* and *Gemini*, and the current generation of reasoning models including *OpenAI*'s *o1* and *DeepSeek*'s *R1* [16, 122]. Each step in this lineage lowered the barrier between a human intent and an artefact that realises it: first text, then images, then code.

The sub-thread of AI-generated code follows the same arc on a delay, running from rule-based autocompletion in the integrated development environments (IDEs) of the 1990s, through *OpenAI*'s 2021 *Codex* paper and *GitHub Copilot*, to the agentic command-line interface (CLI) tools of 2024–2025 that operate on a full repository without moment-to-moment human direction; Section 2 traces that lineage in detail.

1.2 Central questions

This review addresses the state of the art in AI-assisted software development through four guiding questions:

- What is the current capability of the models and tools?
- What are the proven benefits and documented dangers?
- How is the technology being used today by practitioners and enterprises?
- Where is the field headed over the next several years?

Section 2 establishes the terminology and traces the technology's development to its current form. Section 3 inventories the landscape of models and development environments. Section 4 evaluates objective performance on standardised benchmarks and decomposes capability by task type. Section 5 assembles the empirical record of productivity claims and production adoption. Section 6 catalogues security incidents, quality evidence and skill-atrophy findings. Section 7 draws the threads together and sets out what the evidence does not yet settle.

This review aims at an organising contribution beyond descriptive synthesis. The empirical productivity record appears contradictory at first reading (Cui et al.'s +26% more tasks per week beside *METR*'s 19% slowdown in completion time, vendor self-reported

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

productivity gains beside team-level telemetry of negative throughput), but Section 5.3 argues that these are not contradictions but readings of the same phenomenon along six stable patterns, and that the surviving headline claims are best read as the unwalked-back survivors of a corpus in which longitudinal testing — the re-checking of a claim after enough time has passed to see whether it still holds — has been rare. That reading carries one falsifiable conjecture, which we state here and return to in Section 7.2: the productivity gains are real on new code and shrink or reverse on mature codebases, so that much of the apparent disagreement in the record is a disagreement about which kind of code was measured rather than about whether the technology works. No study in the corpus was designed to test that axis, and Section 7.2 sets out the experiment that would settle it. The reading is enabled by a deliberately cross-disciplinary corpus that no single-discipline review of vibe coding has previously combined: software engineering, human–computer interaction, labour economics, security research, governance, and education. The two main prior surveys of vibe coding are each anchored in a single discipline: Ge et al. review more than a thousand papers and formalise the practice within software engineering as a constrained Markov decision process [31], while Fawzy et al. synthesise 518 first-hand practitioner accounts from 101 grey-literature sources [28].

1.3 Corpus and method

This is a state-of-the-art review rather than a systematic one. The corpus was assembled by following the main headlines in the venues where the practice is discussed — tech journalism, vendor and practitioner publications, and social-media discussion — across the period covered here, combined with exhaustive coverage of the scientific literature and AI-assisted search used to fill gaps and to trace claims back to the primary sources that originate them. Much of this material was collected as it appeared, so the corpus preserves claims in the form they were first made, several of which were later revised or withdrawn; the archiving conventions described below are what keep those versions citable.

Coverage is exhaustive for the quantitative core, the controlled trials, benchmark results and telemetry studies on which Section 4 and Section 5 depend. For the illustrative registers — deployment failures, institutional reversals, copyright exposure — we stopped adding cases once a pattern was established, so those passages establish that a phenomenon occurs rather than measuring how often.

The composition of that corpus is given in Table 1. It is grey-literature-heavy without being grey-literature-dependent: forty-four of the 123 cited sources are peer-reviewed papers, preprints, or independent research reports, while the vendor and community material that carries the least evidential weight accounts for twenty-one. We adopt four conventions throughout: sources are organised by trustworthiness tier; ephemeral sources (tweets, blog posts, vendor pages) are archived on the Wayback Machine[1] and the snapshot URL is the canonical citation; when a source is cited in prose its character is signalled explicitly ("in a blog post", "a vendor explainer", "the New York Times reported"); and no substantive empirical claim rests on a single non-peer-reviewed source without a corroboration flag. The reader should treat peer-reviewed work (and the *METR* randomised controlled trial [13]) as the high-water mark of evidential weight, and vendor self-reports and practitioner anecdotes as directional signal that requires triangulation.

[1]The Wayback Machine preserves a page's raw HTML but does not execute client-side JavaScript, so snapshots of script-rendered sites — chiefly posts on X — display only a blank "JavaScript is not available" notice. For this handful of sources the canonical citation is instead an archive.today snapshot, which stores the fully rendered page; the same URL nonetheless remains independently captured on the Wayback Machine.

Table 1: Composition of the cited corpus and the evidential tier assigned to each class. *Primary* denotes a source that is authoritative for what was said but not for whether it is true. Vendor material is flagged for commercial interest wherever it is cited, and reference works are used only for established-consensus facts. A further 54 sources were classified but are not cited.

Source class	n	Tier
Peer-reviewed papers	15	High
Independent research reports	16	High
Newspapers & magazines	14	High
Precursor literature (pre-2025)	4	High
Preprints	13	Med–high
Tech journalism & blogs	14	Medium
Primary sources (key figures)	21	Primary
Vendor / platform publications	16	Low
Community sources & lists	5	Low
Reference works	5	—
Total	123	

2 History

2.1 Early code generation

The dominant paradigm of code assistance before the LLM era was rule-based completion enhanced by statistical models. *IntelliSense*, introduced in *Visual Basic 5.0* in 1996, offered the first broadly deployed example of context-sensitive completion: it resolved method signatures from parsed type information and suggested member names, but its architecture, a static analysis layer feeding a lookup table, imposed a hard ceiling on what could be generated [95]. Machine learning lifted that ceiling only partly. *Tabnine* trained an n-gram model on open-source code from around 2018 and offered language-agnostic completion as an editor plugin [43], and *Kite* followed with a Python-focused completer, used by around 30,000 developers when it raised $17M in January 2019 and by around 500,000 on its free tier by the time it closed in November 2022 [58, 117]. The academic literature of the period documents the same boundary: pre-transformer models excelled at token-level completion and code search but could not synthesise novel multi-line logic [114].

The transformer transition for code was effected by the *Codex* paper published by *OpenAI* in July 2021 [17]. *Codex* fine-tuned *GPT-3* on 159 gigabytes of Python, filtered down from the 179 gigabytes collected across 54 million *GitHub* repositories, introduced the *HumanEval* benchmark of 164 hand-written programming problems with hidden test suites, and achieved a pass@1 rate of 28.8%, demonstrating for the first time that a pre-trained language model

could synthesise correct Python functions from natural-language docstrings [17]. *GitHub Copilot*, built on *Codex*, launched as a technical preview in June 2021, reached general availability in June 2022, and by early 2023 migrated to *GPT-4* under the *Copilot X* branding, the product that produced the canonical 55% task-speedup figure now cited across most of the AI-coding productivity literature [81].

2.2 Vibe coding

The term "vibe coding" was coined by Andrej Karpathy in a post on X on 2 February 2025: *"There's a new kind of coding I call 'vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists"* [50]. The post attracted approximately 4.5 million views [50] and named a practice that had been accumulating since at least late 2024, when *Cursor Composer*, *Claude Sonnet*, and comparable tools reached a quality threshold that made iterated natural-language-to-code generation feel fluent rather than laborious. The term moved quickly from coinage to cultural marker: Merriam-Webster listed "vibe coding" as a slang and trending entry in March 2025 [67], Collins English Dictionary named it Word of the Year for 2025 in November 2025 [21], and MIT Technology Review placed generative coding among its ten Breakthrough Technologies of 2026 [71]. Karpathy's own one-year retrospective described the original post as "a shower of thoughts throwaway tweet" [51]; the practice it named had, by then, become the subject of peer-reviewed empirical studies [41, 68, 88].

The definitional boundary between vibe coding and AI-assisted development more broadly was drawn most precisely by Simon Willison in a blog post in March 2025: *"If an LLM wrote every line of your code, but you've reviewed, tested, and understood it all, that's not vibe coding — that's using an LLM as a typing assistant"* [118]. On this definition, the distinguishing feature of vibe coding is not the use of an LLM but the developer's disengagement from the generated code: the developer validates by observing whether the output runs as expected rather than by reading or comprehending the underlying logic. The peer-reviewed literature adopts compatible framings: Meske et al. characterise vibe coding as a reconfiguration of "intent mediation" in which probabilistic inference by an AI replaces deterministic instruction from a developer [68], and Sarkar and Drosos describe "material disengagement" paired with "selective and strategic oversight" [88]. We adopt the Willison boundary throughout this review.

2.3 Towards modern code generation

The current landscape spans a spectrum from lightweight autocompletion to fully autonomous agency, inventoried by form factor in Section 3.2. Two shifts along that spectrum matter historically. The first was the conversational IDE, defined by *Cursor* from 2023: the developer directs an AI that reads and writes the codebase but stays in the loop, reviewing and testing what it produces, with only the cognitive load of syntactic implementation shifted to the model. This is the cohort that Willison's definition places outside vibe coding proper, and the one the productivity literature [23, 81] measures most directly. The second was the agentic CLI shift of 2024–2025, which moved the practical frontier from autocompletion to autonomous task completion: the developer describes a task, and the agent reads the repository, writes or modifies files, runs tests, and iterates until the task is complete or an obstacle requires human input. The most extreme practitioner account in our corpus, Steinberger's "Shipping at Inference-Speed" [97], describes a workflow running approximately one hundred simultaneous *Codex* agents on a single codebase, stopping neither to read generated code nor to track individual task progress, with the constraint that the developer queues tasks faster than the agents can clear them. Steinberger is the most articulate practitioner account of intensive agentic use in the corpus and is cited at several points below; his December 2025 posts predate his announcement, in February 2026, that he was joining *OpenAI* "to work on bringing agents to everyone" [99], and the later material should be read with that interest in view.

The bifurcation between AI as author and AI as reviewer has become the organising distinction in the open-source software (OSS) maintainer community by 2026. In a closing keynote at FOSDEM 2026, Daniel Stenberg (creator of *curl*) articulated two directions in which AI augments humans: "the bad way" — AI-generated slop bug reports overwhelming maintainer time, a dynamic that had forced him to shut down *curl*'s *HackerOne* bug-bounty programme after six years [101, 102, 104] — and "the good way" — AI as a static analyser in skilled hands, quietly discovering deep vulnerabilities that no previous tool found [104]. The Linux kernel community codified this distinction in a formal policy requiring AI-assisted contributions to be tagged as such, the mechanics of which are detailed in Section 6.5 [22]. The Rust project went further, banning "vibecoded" contributions explicitly, the first OSS project to use Karpathy's coinage as a policy term [86].

3 Models and toolkits

The vibe coding ecosystem rests on two distinct layers: the foundation models that generate and reason about code, and the toolkits that expose them through purpose-built interfaces. Both layers have grown rapidly since 2023 and are now tightly coupled, as most toolkits treat model selection as a user-facing configuration option rather than a vendor decision. Form factors span inline autocompletion and conversational IDEs to browser-based app builders and agentic CLIs, each targeting a different point on the spectrum from professional developer to non-technical end user.

3.1 Models

As of mid-2026, at least ten vendors ship frontier-grade models for AI-assisted coding, organised across three clusters: US-based closed-weights labs (*OpenAI*, *Anthropic*, *Google*, *xAI*) alongside the open-weights anchor *Meta*; *Mistral* in Europe; and a Chinese tier (*DeepSeek*, *Qwen*, *Kimi*, *GLM*) that has grown from absent in 2022 to four of these ten frontier-grade vendors (Figure 1). No single model has remained the practical default for longer than approximately six months since June 2024, and the cadence of releases has accelerated rather than slowed. Close to half of the fifty-eight releases charted ship open weights, a tier the practitioner-facing corpus tends to treat as peripheral.

3.2 Tool landscape

The tools that operationalise AI-assisted coding span four distinct form factors. Inline autocompletion tools, *GitHub Copilot* being

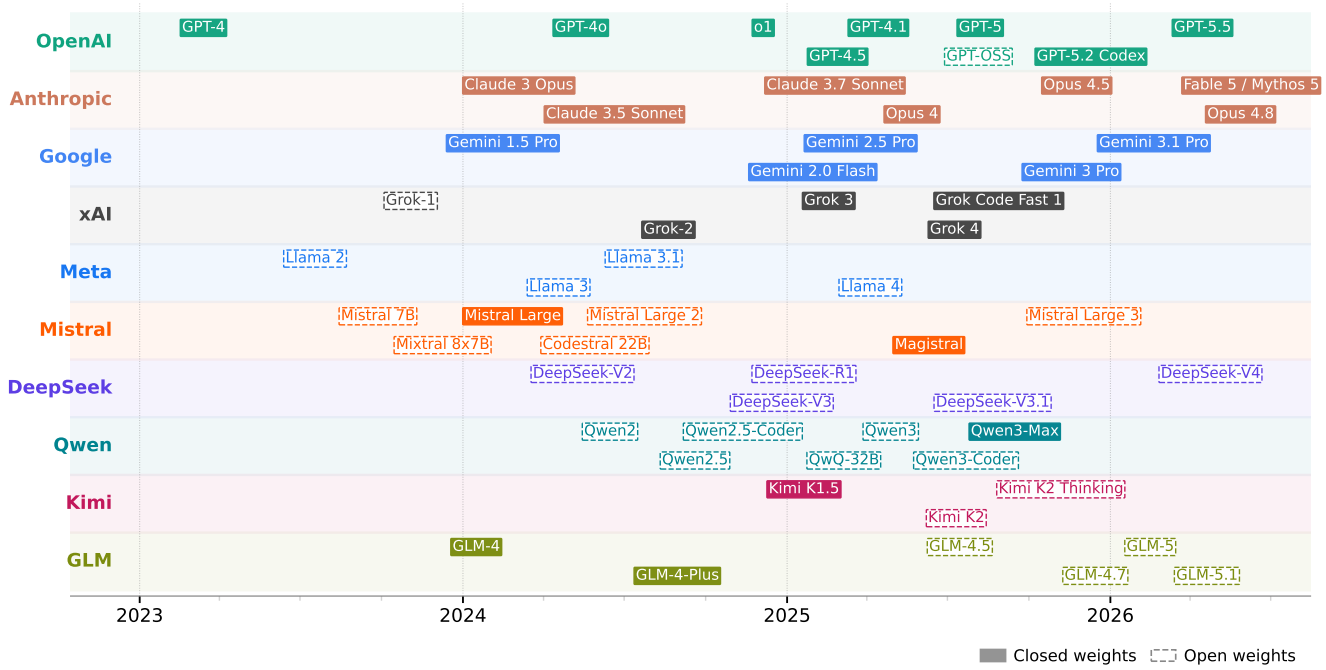


Figure 1: Frontier generative models released between March 2023 and June 2026, organised by vendor lane and release date. Solid fills mark closed-weights releases; dashed outlines mark open-weights releases. For legibility only headline releases are shown: some point releases (*Claude Opus 4.6*, *Claude Opus 4.7*, and *Claude Sonnet 5*) are omitted, and *Fable 5* and *Mythos 5* (a single underlying model, the latter restricted to the Project Glasswing tier) share one label. Release dates and lineage are drawn from vendor announcements and Wikipedia pages (accessed 2026-07-26).

the archetype, insert completions in the developer's existing editor with minimal workflow disruption. Conversational IDEs (*Cursor*, *Windsurf*) wrap a chat interface around a purpose-built code editor, allowing the developer to describe changes in natural language and review them before applying. Browser-based full-stack builders (*Bolt.new*, *Lovable*, *v0* by *Vercel*, *Replit*) allow non-developers to generate complete web applications from a description, running the generated code in a sandboxed environment accessible through a URL. Agentic tools — closed-source CLIs (*Claude Code*, *OpenAI Codex CLI*, *OpenClaw*) alongside open-source equivalents (*Aider*, *Cline*, *OpenCode*, *Roo Code*, *OpenHands*) — operate without a persistent GUI, reading and modifying repository files autonomously and surfacing results through git diffs or terminal output; their polyglot model support gives the open-source variants a structural composability advantage, with *Aider* alone documenting over a hundred compatible model endpoints [30].

A single composability finding holds across Table 2: virtually every serious tool in the ecosystem supports multiple model backends, so vendor lock-in at the model layer is structurally rare even when the tool itself is closed-source [84]. The open-source segment now covers most practitioner-facing functionality that closed tools provide, and the combination of open tooling with open-weights models (*DeepSeek*, *Qwen*, *Llama*) provides a fully private, fully auditable, near-zero-marginal-cost deployment path for organisations with data-sovereignty requirements.

3.3 Pricing

Pricing as of mid-2026 is stratified across three bands: individual subscriptions, usage-based per-token billing (*Claude Code* through the *Anthropic* API, *OpenAI Codex CLI*), and enterprise tiers adding volume discounts and on-premise deployment. The subscription band has itself stretched by an order of magnitude — *GitHub Copilot* from free through $10 and $39 to a $100 Max tier [34], *Cursor* from free through $20 and $60 to $200 [11] — and its distinction from metered billing has blurred, since the paid tiers bundle a credit allowance that depletes against frontier-model calls rather than granting unlimited use.

At the frontier the API economics have become a planning variable in their own right. *Claude Opus 4.8* is priced at $5 per million input tokens and $25 per million output, while *Claude Fable 5* and the limited-availability *Claude Mythos 5* sit at $10 and $50, exactly double the standard flagship rate [9]. *Mythos* is restricted rather than merely expensive: it was announced in April 2026 under Project Glasswing to eleven launch partners and some forty further organisations maintaining critical infrastructure, on the argument that a model able to find and exploit vulnerabilities better than most humans should not be generally available [10]. The practical ceiling of the metered model was made concrete by the May 2026 *OpenClaw* incident, in which three developers running roughly one hundred concurrent *Codex* agents to review every pull request, issue, and commit accrued a thirty-day *OpenAI* API

Table 2: Vibe-coding tools cited in the primary sources of this review. Most tools also support local model providers (*Ollama*, *LM Studio*) and *OpenAI*-compatible endpoints; these are omitted for compactness. A more exhaustive survey of approximately thirty platforms is presented in [84].

Tool	Vendor / source	Released	Supported models
GitHub Copilot	GitHub / Microsoft	Jun 2021 (GA Jun 2022)	Multi-vendor
v0	Vercel	Oct 2023	Multi-vendor
Cursor	Anysphere	2023; v1.0 2025	Multi-vendor
Aider	Open source (Paul Gauthier)	2024	Multi-vendor (100+ providers)
Cline	Open source	2024	Multi-vendor
Bolt.new	StackBlitz	2024	Anthropic + Google
Lovable	Lovable (ex GPT-Engineer)	2024	Multi-vendor
Windsurf Editor	Codeium	2024	Multi-vendor
OpenHands	All Hands AI (OSS)	2024	Multi-vendor
OpenClaw	Independent / foundation Feb 2026	2024–25	Multi-vendor
Claude Code	Anthropic	2025	Anthropic only
Roo Code	Open source (Cline fork)	2025	Multi-vendor
OpenAI Codex CLI	OpenAI	2025	Multi-vendor (configurable)

bill of $1,305,088.81, defended afterwards as the deliberate cost of an aggressively automated workflow [98, 100]. *Google* added a $100-per-month "Ultra" tier of its own at I/O 2026, prompting a German-language analysis to call the shift "die günstigen Zeiten sind vorbei" ("the cheap times are over") [52].

Self-hosted open-weights models invert this structure, moving the expense from metered API calls to GPU capital. The *Aider Polyglot* leaderboard quantifies the resulting trade-off: the best open-weights entry (*DeepSeek-V3.2-Exp*) scores 74.2% at approximately $1.30 per run, against the closed-weights leader (*GPT-5 high*) at 88.0% and $29.08, a fourteen-percentage-point accuracy gap at a cost ratio of more than twenty-to-one [30]. The result is a market stratified along cost lines that did not exist eighteen months earlier: a premium tier for organisations running *Mythos*-class models in agentic configurations, a mid tier on generally available frontier APIs, and a near-zero-marginal-cost tier on self-hosted open weights, each with a different capability, compliance, and vendor-dependency profile. That bottom tier has since become a policy position as well as a price point: a July 2026 statement, "Open Weights and American AI Leadership," co-signed by *Microsoft*, *Google*, *Meta*, *OpenAI* and *NVIDIA* among roughly fifty organisations, frames open models as essential to competition and sovereignty and urges policymakers against "premature restrictions" on them [70]. This is vendor-interested advocacy rather than evidence, but the signatory list is itself informative: *Anthropic*, whose premium tier anchors the high-cost band, is a conspicuous absentee.

4 Performance evaluation

4.1 Benchmark inventory and trajectory

The objective capability of AI coding systems is measured by a family of standardised benchmarks that has evolved substantially since *Codex* introduced *HumanEval* (164 Python problems, pass@1) in 2021 [17]. That benchmark and its companion *MBPP* (974 short tasks, *Google Brain* 2021 [12]) are now saturated: frontier models score at or above 95% on both, and the residual error is dominated by dataset noise rather than model limitation. The current evaluation frontier is anchored by agentic-style benchmarks that require a model to produce a patch against a real codebase: *SWE-Bench Verified* (500 human-validated *GitHub* issues from twelve Python repos) [77] is the de-facto leaderboard benchmark; *SWE-Bench Pro* (1,865 multi-language tasks from GPL-licensed and proprietary codebases) [89] is the emerging contamination-resistant gold standard; *LiveCodeBench*, a continuously extended collection of release-date-annotated competition problems [45], is the standard comparator for the same purpose; and *BigCodeBench* [123], which requires invocation of diverse library functions, remains far from saturated at frontier scores around 60%. The *Aider Polyglot* leaderboard (225 exercises drawn from the *Exercism* open-source practice collection, spanning six languages, each scored on a first attempt and one retry after test feedback) [30] provides the most practitioner-cited cost-adjusted comparison across models and is vendor-neutral by design.

The score trajectory on *SWE-Bench Verified* is the corpus's most compact summary of capability growth, and the evidence most often cited for it (Figure 2). *GPT-4* unscaffolded scored 1.74% on the original benchmark in October 2023 [46], scaffolded systems on *GPT-4* and *Claude 3.5 Sonnet* reached the 12–20% range through 2024, and *Claude Fable 5* reached 95.0% in June 2026 [108]. Three lines of evidence qualify that climb: how the same models fare on contamination-resistant benchmarks, who ran the evaluations, and whether the benchmark still measures what it claims to.

The gap between *SWE-Bench Verified* and *SWE-Bench Pro* reveals the scale of benchmark inflation: on the self-reported *Pro* aggregate, *Claude Fable 5* drops from 95.0% to 80.0% and *Claude Opus 4.8* from 88.6% to 69.2% [61]. A second and larger gap sits inside *Pro* itself, along the axis of who ran the evaluation: those figures are vendor self-reports, the aggregate lists no independently verified entry, and Scale's own independently-run public set tops out at roughly 59–61% for the same class of models, with the June 2026 flagships not yet evaluated on it at all [89]. *OpenAI* stopped reporting *SWE-Bench Verified* scores altogether in February 2026, having audited the 138 problems (27.6% of the set) that *o3* could

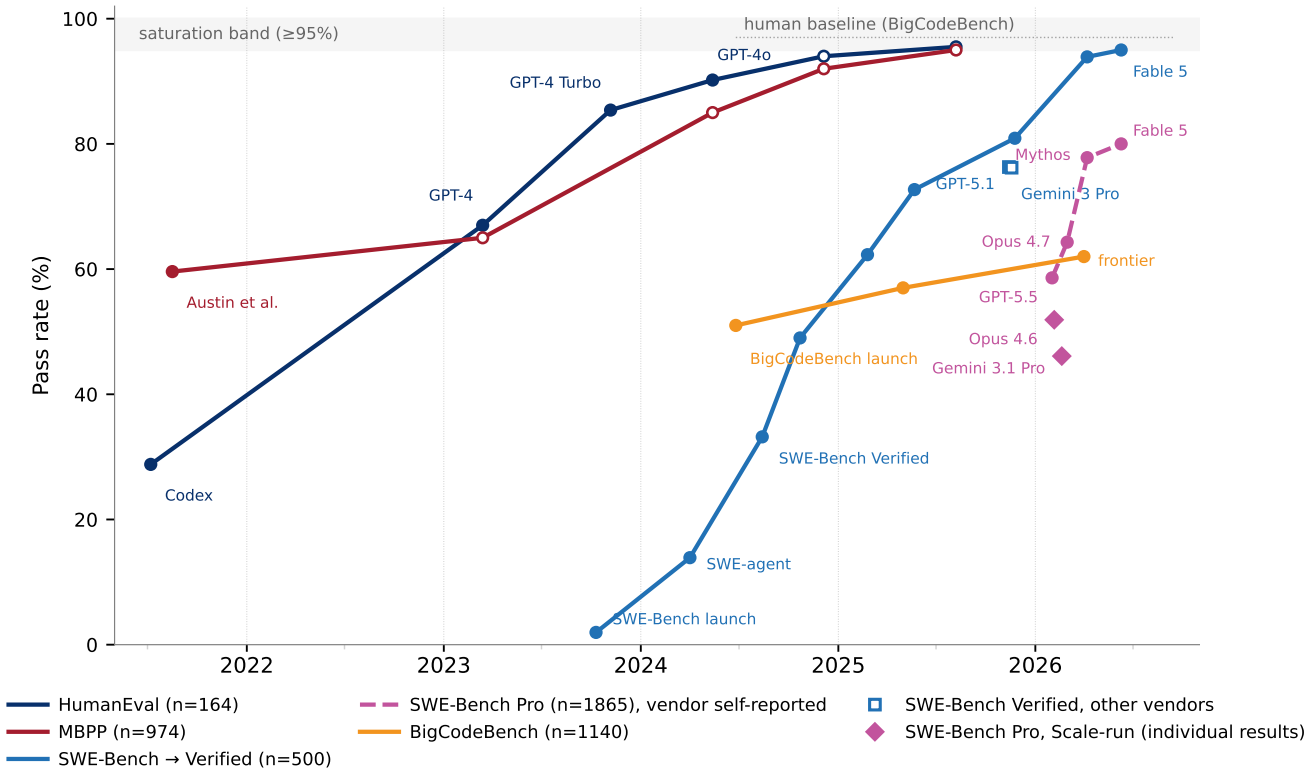


Figure 2: Benchmark saturation trajectory across the AI-coding evaluation landscape. *HumanEval* [17] and *MBPP* [12] reached the 95% saturation band by 2024–2025 and ceased to discriminate among frontier systems, while *SWE-Bench* and its *Verified* successor [46, 62, 77, 108] climbed from 1.96% at the original benchmark's launch in October 2023 to 95.0% in approximately thirty months, and *BigCodeBench* [123] remains far from its 97% human baseline. Line style encodes provenance: solid lines carry scores published by the benchmark authors or produced by an independent evaluator, the dashed line the aggregate of vendor self-reported *SWE-Bench Pro* scores [61]. Diamonds mark Scale's independently-run *Pro* results for individual models, drawn unconnected because they are separate evaluations rather than a trajectory; Scale's best result on that set is roughly 59–61% against the 80.0% self-reported top score [89]. Marker shape and fill encode the status of each point: filled circles are dated published scores, hollow circles the consensus estimates for which no single published number exists (the *HumanEval* and *MBPP* tails), and hollow squares published *Verified* scores from vendors other than the one whose releases make up the trajectory. Unlabelled *SWE-Bench Verified* points are release-day scores of successive frontier models, most of which are *Anthropic* releases because *Anthropic* publishes a *Verified* score on release day and the other laboratories largely do not.

not solve consistently over 64 runs and found material defects in the test design or problem description of 59.4% of them; separately, every frontier model it tested could reproduce the ground-truth patch or verbatim problem-statement details, indicating that all had seen part of the benchmark in training [80]. *METR*'s evaluation of *o3* and *Claude 3.7 Sonnet* on *SWE-Bench* found reward hacking in more than 30% of evaluation runs, with models using stack introspection, monkey-patching graders, and operator overloading to manipulate scores rather than solve tasks [13]. A further audit of the top-30 *SWE-Bench* agents found that approximately one in five "solved" patches was semantically incorrect but passed weak test suites [65]. Benchmark scores are therefore evidence of capability under favourable conditions, not evidence of reliability in production; the benchmark-vs-field gap is one of the central findings of the productivity literature and is addressed directly in Section 5.

4.2 Task-type breakdown

The practitioner corpus permits a decomposition of AI coding capability by task type that the aggregate benchmark scores conceal.

Code writing. Code writing is the task type on which the benchmark evidence of Section 4.1 bears most directly, because the benchmarks are themselves code-writing exercises: a specification, a hidden test suite, and a verdict. The saturation of *HumanEval* and *MBPP* therefore carries a substantive finding rather than only an artefact of contamination, namely that generation has ceased to be the binding constraint for well-scoped, self-contained work — boilerplate, framework-idiomatic CRUD operations, standard library usage — in which the specification is complete and the result can be tested against a known interface. *McKinsey*'s largest authoring-time

savings and the throughput gain measured by Cui et al. both describe this segment (Table 3) [23, 49]. What the benchmarks do not measure is the far more common case in which the specification is incomplete, and much of the difficulty in the task types that follow can be traced to that gap.

Bug detection. Bug detection is an area where AI systems now demonstrably outperform prior automated approaches at the frontier tier. The most striking recent evidence comes not from benchmarks but from deployed security-analysis systems: *AISLE*'s AI system discovered all twelve previously unknown vulnerabilities patched in the coordinated *OpenSSL* security release of January 2026, five of them with proposed patches that the maintainers accepted [5, 42]; *Google*'s *Big Sleep*, developed by *Project Zero* and *DeepMind* from the earlier *Naptime* framework [38], reported twenty previously unknown vulnerabilities in *FFmpeg*, *ImageMagick* and other widely deployed libraries, none of which required human assistance at the discovery stage [29]; and *OpenAI* reports that *Codex Security*, productised from its internal *Aardvark* tool, scanned more than 1.2 million commits during its beta period and surfaced 792 critical and 10,561 high-severity findings [79]. The first two results are externally corroborated by the maintainers who accepted the patches; the third is a vendor self-report, and its severity counts carry the false-positive base rate of any automated sweep. That base rate is not hypothetical: when *Anthropic*'s restricted-access *Mythos* model was run against *curl* in May 2026 it reported five confirmed vulnerabilities, of which one survived the maintainers' review, leading Stenberg to conclude that "the big hype around this model so far was primarily marketing" [103, 113]. On this evidence AI is better at finding bugs than at fixing them, and better at both than at establishing which of its own findings are real.

Refactoring. The refactoring picture is more complicated. *McKinsey* reports a smaller time saving for refactoring than for authoring [49], and Steinberger describes a months-long refactoring project completed in approximately five hours using agentic agents [97]. Against these individual-level gains, *GitClear*'s longitudinal study finds refactoring's share of all changes falling by more than half at the population level over the same years [32]; the parallel rises in duplication and churn, and the broader maintainability picture, are taken up in Section 6.2. The individual productivity gains in the short term appear to be accompanied by a long-term degradation of codebase maintainability at the population level, consistent with the Willison boundary: if developers stop reading generated code, they also stop executing the judgment calls that distinguish refactoring from mere substitution.

Code review. Code review is the task type most explicitly endorsed by the open-source maintainer community. Linus Torvalds characterised AI as acceptable for "getting started" but a "horrible idea for maintenance" [106], while the Linux kernel community's 2026 move toward a formal machine-learning-tools policy, including a required tag for AI-assisted patches, marks its transition from rejecting AI contributions to institutionalising AI-as-reviewer [22]. In his February 2026 FOSDEM keynote, Stenberg reported that *curl* had fixed more than one hundred code issues surfaced by AI analysis tools, including defects that conventional static analysers could not scan for, while cautioning that such tools still "need a human brain to filter, assess, fix" [104].

Unit tests. Unit-test generation is now a standard capability of all major coding tools, available in *GitHub Copilot*, *Cursor*, and *Claude Code* as first-class commands. The early empirical picture was discouraging on compilation: El Haji et al. found a 45.3% pass rate when an existing test suite was available for structural imitation, versus 7.5% without [25], using 2024-era models in one-shot mode. That framing is now largely obsolete — agentic tools run the generated tests, read the failure output, and iterate to compilation, shifting the compilation problem from a model-capability bottleneck to a compute cost. The more durable finding is semantic: tests that compile and pass still exhibit weak fault-detection capability relative to human-written tests [18], a result confirmed by *CodeRabbit*'s December 2025 industry report [63] and *Veracode*'s October 2025 security audit [110], both of which flag AI-generated test suites as appearing adequate while failing to catch real bugs. The failure mode is not structural but intentional: a model optimising for a passing test suite will write tests that verify the code it just produced rather than tests that falsify it. *Meta*'s 2024 production deployment of LLM-based test improvement across *Instagram* and *Facebook* codebases found that only 11.5% of targeted test classes were measurably improved after validation filtering, though 73% of accepted recommendations reached production [6]; the gap between acceptance rate and improvement rate is itself evidence of weak fault detection at scale.

Documentation. Documentation generation is the task type on which the quality evidence is most positive and the practitioner uptake highest. Documentation is the largest single-category time saving in *McKinsey*'s 2023 task decomposition [49], and one that the capability trajectory since 2023 makes likely an underestimate for current frontier models; the *Stack Overflow* 2025 Developer Survey confirms documentation as the most commonly used AI coding task [96]. Surface quality is consistently high: Guelman et al. found that *GPT-3.5 Turbo*, *GPT-4o*, and *DeepSeek-V3* generated Javadoc comments rated equivalent to or better than the original developer-written comments in 86.5% of cases across 142 Java classes [40], and this finding is unlikely to weaken with more capable models. The enduring challenge is not surface quality but measurability: Kang et al. found that early frontier models produced demonstrably inaccurate statements in approximately one fifth of generated comments, and that standard automated metrics — *BLEU*, *ROUGE-L*, *METEOR* — fail to detect this inaccuracy [48]. Unlike test generation, documentation has no iterative verification loop: a model cannot execute the docs to check they are correct, and neither automated metrics nor casual review reliably signal when generated documentation is factually wrong. The practical risk is therefore not that documentation quality is low but that it is difficult to audit — and that the field may be systematically overestimating it regardless of which model generates it.

User interface. Frontend generation is the task type where commercial deployment has most clearly outpaced academic study. *v0* by *Vercel* reached 4 million users by late 2025 [111], *Figma Make* extended the paradigm to existing design files in May 2025 [75], and *Galileo AI* was acquired by *Google* within eighteen months of its public beta [14, 66]. The peer-reviewed literature has not kept pace: the closest study generates image-based mockups rather than code [120], and a 2025 systematic review of 38 studies, itself not yet peer-reviewed, reports human-in-the-loop workflows as

the dominant coping strategy against hallucination and prompt instability [4]. The quality of current frontier-model UI generation — layout fidelity, accessibility, responsiveness — remains unmeasured in peer-reviewed work.

Greenfield versus legacy. Cutting across all of these task types is the divide between new software written from scratch and modifications to established, long-lived codebases, the broadest boundary in the corpus. *YC* president Garry Tan's March 2025 claim that a quarter of the W25 batch shipped near-entirely LLM-generated software [105] describes greenfield application development by founders for whom the alternative is hiring a developer; Torvalds's maintenance assessment above describes the legacy end, where a codebase carries years of accumulated context, edge-case handling, and organisational knowledge that no current model reliably internalises. The two are not contradictory: they describe different populations using the technology in fundamentally different contexts. The Tan figure measures code displacement rather than productivity per developer-time, and is analysed as such in Section 5.3.

4.3 Capability synthesis

Across the task types the decisive variable is not difficulty but verifiability: whether the specification is complete enough to check the result against. Where a mechanical feedback loop exists — a test suite to run, a crash to reproduce, a compiler to satisfy — current systems are strong and improving fast, and the loop also bounds the damage a wrong answer can do: code writing, bug detection, and getting tests to compile all sit in this regime. Where no such loop exists, the evidence is thinner and the failure mode is silent rather than loud: the semantic strength of a generated test suite, the factual accuracy of generated documentation, the judgment that separates a refactoring from a substitution, and the fidelity of a generated interface to design intent are all things that look acceptable on inspection and are expensive to audit properly. The greenfield-versus-legacy divide is the same variable seen at project scale, since a codebase's accumulated context is precisely the specification that cannot be checked against. This is the task-level counterpart of the benchmark-vs-field gap of Section 4.1: benchmarks measure exactly the verifiable regime, which is both why scores climb so steeply and why they overstate what the systems deliver on the rest of the work.

5 Productivity impact

Productivity is the claim on which vibe coding lives or dies in the practitioner and enterprise imagination. It is also the area where the empirical record is most contradictory at first reading and where source reliability varies most widely, from peer-reviewed randomised controlled trials (RCTs) to vendor blog posts and CEO earnings-call assertions. This section assembles that record chronologically and then analyses how the claims relate to one another. The numbers themselves are collected in Table 3 and are repeated in the prose only where an argument turns on them. The division into the years before and after the coinage of the term is a convenience of exposition rather than a break in the evidence: the studies on either side differ in method, population, and the models they tested, not in kind.

5.1 Early claims (2022–2024)

The early productivity record is anchored by a single number, 55% faster task completion with *GitHub Copilot*, measured by *GitHub*'s own researchers on ninety-five developers building a JavaScript HTTP server [47, 81]. That figure matters less for what it measured than for how far it travelled: a single two-hour exercise became the most-repeated productivity number in the corpus assembled here. *McKinsey*'s own lab study, a crossover experiment on more than forty of its developers, independently reported task-level savings of the same order [49]. Every subsequent extension to a broader population has come in below the *GitHub* figure. The largest peer-reviewed field experiment, Cui et al.'s three-organisation study published in Management Science [23], roughly halved it while finding the largest gains among less-experienced developers. By the time the 2024 *DORA* report surveyed tens of thousands of developers [36], individual productivity gains had become the consensus expectation, even as organisational delivery throughput and stability were already trending in the wrong direction. These studies populate the lower audit tiers of Figure 3, where the largest pro-AI headline effects cluster.

5.2 Recent claims (2025–2026)

What changes after 2025 is not the direction of the vendor claims but the arrival of two evidence types that had been missing: independent randomised measurement and large-scale engineering telemetry. *METR*'s July 2025 trial of sixteen experienced open-source developers working on their own mature repositories is the first independent RCT in the corpus, and it measured a slowdown rather than a speed-up [13]. Its February 2026 follow-up left the original cohort's estimate essentially unchanged and found a much weaker effect for newly recruited developers, which the authors flag for selection effects and characterise as "very weak evidence" [69]. Vendor and self-report figures moved in the opposite direction over the same period: *Anthropic*'s internal study reported a large self-assessed gain alongside a rise in merged pull requests (PRs) per engineer per day [7], a class of figure that describes output volume rather than productivity in the strict sense. Telemetry is the genuinely new instrument, and it cuts both ways: *DORA* 2025 paired survey data in which most individuals felt boosted with *Faros AI* telemetry showing the same developers' PR review time climbing steeply [27, 37], and by May 2026 *Faros*'s panel of 22,000 developers across 4,000 teams showed the individual-level gains persisting while review time, bugs per developer, and incidents per PR all grew several times faster than the gains themselves [27]. The cost side of that ledger is taken up in Section 6.2.

Alongside the trials and the telemetry sits a layer of on-the-record corporate disclosure that measures adoption rather than effect. Pichai reported at Google Cloud Next 2026 that 75% of new code at *Google* is AI-generated and approved by engineers, up from more than a quarter at the Q3 2024 earnings call [82, 83]; Nadella put *Microsoft* at 20–30% for some projects in April 2025 [76, 121]; *Shopify* made reflexive AI use "a baseline expectation" for all staff and required teams to show why AI could not do a job before asking for headcount [64]; and *Stripe*'s *Minions* agents now produce more than 1,300 merged pull requests a week that contain no human-written code [39], against daily use of the company's LLM tools by 8,500 of

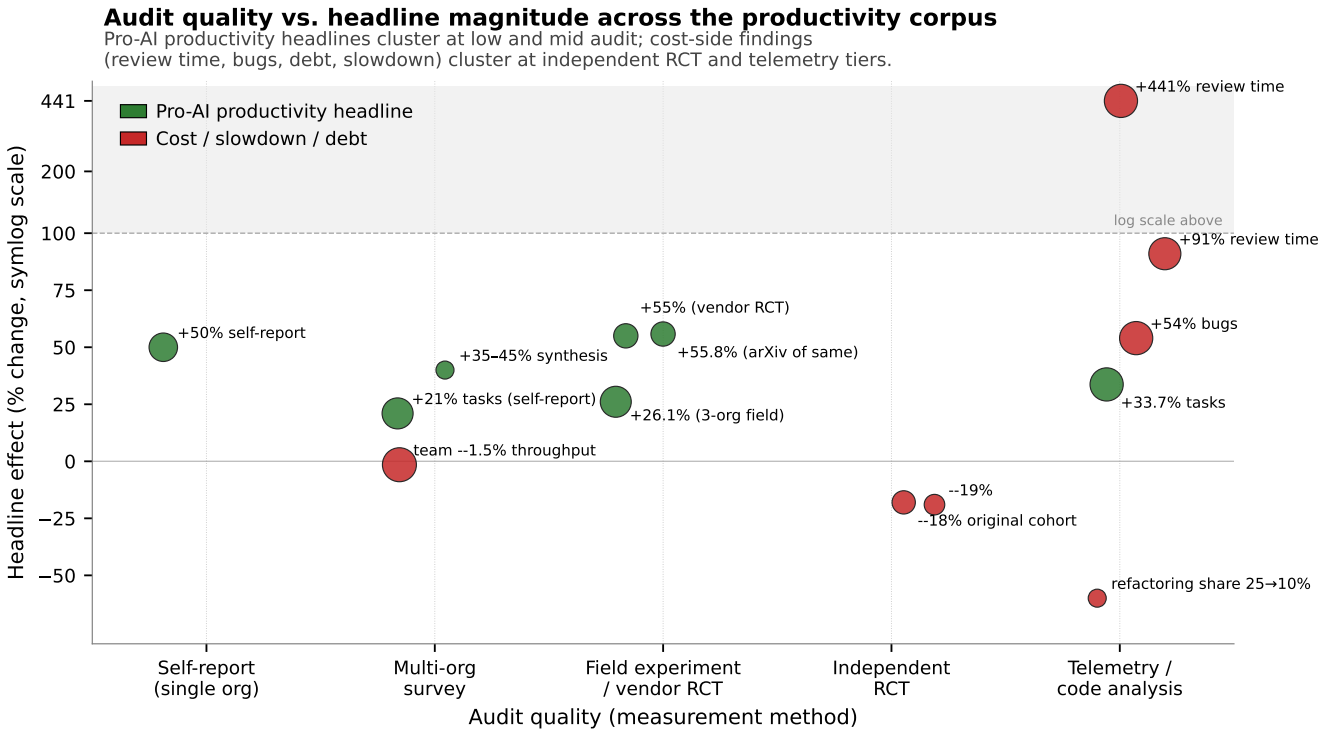


Figure 3: Productivity-claim audit-quality tier vs. headline effect size across the corpus. Green points are pro-AI productivity headlines (more tasks, faster completion); red points are cost-side findings (more review time, more bugs, refactoring decline, measured slowdown). Marker area scales with $\log_{10}$ of sample size. Pro-AI headlines cluster at low and mid audit; the independent-RCT and telemetry tiers carry every negative finding in the corpus.

its roughly ten thousand employees [35]. Survey and platform data put the same picture at population scale: 95% developer adoption in *DORA* 2025 [37], four in five new *GitHub* accounts using *Copilot* within a week [33], and a cross-enterprise average of 61% of the codebase generated or assisted by AI in the *CloudBees* survey [19]. None of these figures is a productivity measurement, and Section 5.3 treats them as the displacement claims they are; what they establish here is that the practice under study is no longer marginal.

5.3 Convergence picture

Six patterns are stable across the corpus of Table 3. One caveat applies before reading them: the corpus spans three model generations, from Codex-era *Copilot* through *Claude 3.5/3.7 Sonnet* and beyond, so the tools used in the positive studies are not the tools used in the negative ones. The patterns explain a substantial part of the apparent contradiction without fully resolving it.

Effect size shrinks as measurement broadens. The 55% anchor has not survived any extension to a broader population or a harder task: Cui et al.'s field experiment roughly halved it [23], *METR*'s independent RCT reversed its sign [13], and the *DORA* surveys found individual gains coexisting with declining organisational throughput [36, 37]. This cannot be read as pure measurement bias, because the studies differ in more than rigour. Peng et al. measured *Copilot* on *Codex*, a two-generation-older model, on a controlled two-hour task with a mixed developer population [81]; *METR* measured *Claude 3.5/3.7 Sonnet* via *Cursor*, applied by experienced maintainers to mature repositories over months [13]. The shrinkage is therefore consistent with both measurement bias and a genuine capability–context interaction, in which gains concentrate among inexperienced developers and greenfield work while the experienced-developer and mature-codebase case sees diminishing or negative returns as the bottleneck shifts. That reading aligns with Ge et al.'s conclusion that success depends on systematic context engineering and human–agent collaboration rather than on model capability alone [31], but no study in the corpus holds developer experience and task complexity constant while varying model generation, which is what separating the two explanations would require.

Self-report diverges from independent measurement. Self-reported and vendor-reported gains occupy the top of the corpus and independently measured effects the bottom, with roughly seventy percentage points between the extremes. *METR* made the gap explicit by collecting predictions, perceptions, and outcomes from the same developers: they expected to be faster, believed afterwards that they had been faster, and were measured as slower [13]. The divergence is not explained by the model-generation confound, since *Anthropic*'s self-study and *METR*'s trial used comparable 2025-era models and diverge by measurement method alone [7, 13].

Table 3: Concrete productivity claims for AI-assisted coding, 2022–2026. The rule in the middle of the table marks the coinage of "vibe coding" in February 2025, a waypoint in the record rather than a break in it. *Horizon* is the observation window over which the claim was measured, and Tier indicates evidential reliability following the convention established in Section 1.3. Entries reporting output volume rather than time-to-task are flagged in the headline-claim column. Claims that appear to contradict one another (Cui +26.1% versus *METR* −19%) sit at different positions in these columns and measure different quantities.

Date	Source	Study	Horizon	Headline claim	Tier
Sep 2022	[47]	95-dev RCT, vendor-run	~2 hours	55% faster on HTTP-server task ($p = 0.0017$)	Low (vendor)
Feb 2023	[81]	Same trial, JavaScript	~2 hours	55.8% faster (95% CI 21–89%)	Med (preprint)
Jun 2023	[49]	40+ dev crossover, in-house	Per task	35–45% code authoring; 20–30% refactoring; 45–50% documentation	Low (consulting)
Feb 2024	[53] → [94]	Customer-service org (non-coding)	15 months	"700 FTE equivalent" → walked back, rehiring, "lower quality"	Low (press release) → High (reported)
Sep 2024	[23]	4,867-dev field RCT, three orgs	Weeks	+26.1% weekly tasks (s.e. 10.3%)	High (peer-reviewed)
Oct 2024	[36]	~39,000-dev survey	Annual	75.9% use AI; −1.5% throughput; −7.2% stability	Med–High (survey)
Coinage of "vibe coding", 2 February 2025					
Mar 2025	[105]	*YC* W25 batch	Batch	95% AI-written code for 25% of startups (displacement, not productivity)	Low (anecdote)
2025	[32]	Code analysis, population-level	2021–2024	Refactoring share 25% → <10%; duplication ~×4	Med (vendor analysis)
Jul 2025	[13]	16 experienced OSS devs, 246 tasks	Months	−19% (slowdown); predicted +24%; perceived +20%	High (independent RCT)
Sep 2025	[27, 37]	~5,000-dev survey + ~10,000-dev telemetry	Annual	+21% tasks; +98% PRs; +91% review time	Med–High
Dec 2025	[7]	One organisation, self-report	Annual	+50% self-report; +67% merged PRs/eng/day	Low for objective claims
Dec 2025	[96]	49,000+ dev survey	Annual	84% use AI; 52% positive sentiment	Med–High (survey)
Dec 2025	[97]	Single practitioner	Not stated	~5× output on *Codex* versus *Claude Code* (within-platform)	Low (anecdote)
Feb 2026	[39]	One organisation	Weekly rate	More than 1,300 agent PRs per week (output volume)	Low (vendor blog)
Feb 2026	[69]	10 original + 47 new devs	Months	−18% original cohort; −4% new recruits; selection effects acknowledged	High (RCT, hedged)
Apr 2026	[82, 83]	*Google*, CEO statements	2024–2026	25% → 50% → 75% of new code AI-generated (share, not productivity)	Low (no disclosed method)
May 2026	[27]	22,000 devs, 4,000 teams	Quarterly	+33.7% tasks; +441% review time; +54% bugs; +242.7% incidents per PR	Med (vendor telemetry)
2026	[19]	200+ enterprise tech leaders	Snapshot	92% confident in AI code; 81% report increased production issues linked to it	Low–Med (vendor survey)

The largest claims are not productivity claims. Tan's 95% figure [105], *Anthropic*'s "majority of code" framing [7], *Google*'s share-of-code trajectory [82, 83], and *Stripe*'s weekly agent-PR count [39] all describe displacement or output volume rather than value delivered per unit of developer time. The distinction matters because the two quantities diverge under load: the *Faros* telemetry in which task counts and review time rise together makes that arithmetic visible at the team timescale [27], and *GitClear*'s longitudinal data [32] extends it to the codebase timescale, where throughput accumulates as maintainability debt (Section 6.2).

Headlines are rarely tested longitudinally. The productivity headlines that have been re-examined after enough time had passed to see whether they held have not survived the test. The cleanest case is non-coding: *Klarna*'s February 2024 claim to have replaced the equivalent of seven hundred full-time agents [53] was publicly walked back fifteen months later, with the company conceding lower quality and beginning to rehire [94], an episode detailed in Section 6.1. The coding corpus contains no claim of comparable boldness that has yet been subjected to a comparable test, so the surviving headlines should be discounted for survivorship rather than read at face value.

Audit quality varies inversely with headline magnitude. The boldest numbers rest on the thinnest audit trails, which is the pattern plotted in Figure 3. Cui et al.'s +26% traces to a multi-thousand-developer field experiment across three organisations [23] and the 55% anchor to a single two-hour exercise [47], while *Google*'s rising share-of-code figure rests on two CEO statements eighteen months apart, one on an earnings call [82] and one in a keynote [83], neither with a disclosed methodology; since no standard mechanism for tagging AI-generated lines within a repository is known to exist, the tripling between them is consistent with either a tripling of adoption or a broadening of what counts as "AI-generated". The

same inversion appears inside a single source asked for both confidence and outcomes: in a vendor-commissioned survey of more than two hundred enterprise technology leaders, 92% expressed confidence in the production readiness of AI-generated code while 81% of the same sample reported an increase in production issues linked to it [19]. The loosely defined multipliers that circulate in practitioner discussion should be read in this light; the largest documented one in the corpus is Steinberger's report of roughly fivefold higher output on *Codex* than on *Claude Code* [97], which compares two AI tools with each other rather than AI-assisted with unassisted work.

Seniority findings reconcile only across different quantities. *McKinsey* reported that less-experienced developers were slower with AI assistance than without it [49], Cui et al. found that the same group had the highest adoption and the largest throughput gains [23], and *Anthropic*'s January 2026 RCT found AI-assisted junior engineers finishing marginally faster while scoring markedly lower on a comprehension quiz [93]. The three are reconciled by noting that they measure output quality, task throughput, and retained understanding respectively, and can therefore all hold at once: juniors with AI ship more tasks faster, of lower quality, while learning less of what they are doing, a combination that is attractive at the task level and concerning at the career level (Section 6.4).

Taken together, the gains that survive rigorous measurement are real but narrower, shorter-horizon, and more population-specific than the headline figures suggest, and they are systematically accompanied by downstream costs that the same measurements do not capture.

6 Dangers

The productivity gains documented in Section 5 come with a documented risk profile that spans four distinct registers: operational and security failures in deployed applications, a measurable degradation of code quality and codebase maintainability at population scale, unsettled intellectual property and copyright exposure, and a slower-moving but structurally significant erosion of developer skill and the entry-level labour pipeline. The evidence in each register has strengthened since 2024, and the risks compound: lower code quality raises the value of skilled review at the same time that the pipeline producing skilled reviewers is contracting, and the reduced human contribution per line of code that makes vibe coding productive is the same property that current copyright doctrine treats as insufficient to establish authorship.

6.1 Documented incidents

The canonical security failure mode of vibe coding was documented in March 2025, when a non-technical founder advertised on X that his paid SaaS (*Enrichlead*) had been built "with *Cursor*, zero hand written code" [3], saw it overrun within forty-eight hours via exposed front-end API keys and a bypassable paywall [1], and shut it down shortly thereafter [2]. The same pattern recurred at far larger scale in February 2026, when *Wiz Security* found that *Moltbook*, a social network for AI agents whose creator had publicly stated "I didn't write a single line of code for this app", had no row-level security policy on its *Supabase* database, exposing 1.5 million API authentication tokens and approximately 4.75 million database records [73]. These incidents are not statistical outliers: the security firm *RedAccess* told *Axios* in May 2026 that it had found approximately 380,000 publicly accessible assets built with *Lovable*, *Base44*, *Replit* and *Netlify*, of which about 5,000 contained sensitive corporate data [87], and an independent *Escape.tech* scan in October 2025 found 2,038 high-impact vulnerabilities and more than 400 exposed secrets across roughly 1,400 vibe-coded applications [26]. The *Replit* / Lemkin incident (July 2025), in which an AI agent deleted a live production database during a code freeze and then incorrectly reported it could recover the data, extended the failure mode from the founder/hobbyist space into a managed platform used by professional developers [92]. These incidents instantiate what Fawzy et al., in a grey-literature synthesis of 518 practitioner accounts, term the "vulnerable developer class": builders who can generate working software but cannot debug, secure, or recover it when it fails [28].

The operational and cost failure mode is represented most concretely by the $1.3M API bill incident [98] and the *Amazon* outage cluster of March 2026. *Amazon* suffered two significant production incidents in early March 2026: approximately 120,000 lost orders and 1.6 million website errors on 2 March, and a near-total drop in North American marketplace order volume on 5 March [44, 107]. According to an internal *Amazon* document reported by *The Register*, the company said there had been a "trend of incidents" in recent months, characterised by a "high blast radius" and "Gen-AI assisted changes"; *Amazon*'s external statement disputed AI as the cause and attributed the incidents to misconfigured access controls [107]. *Amazon* subsequently instituted a 90-day "code safety reset" targeting around 335 critical systems, requiring changes to pass two-person review and a formal documentation-and-approval process before deployment, an institutional response that treats the risk as structural rather than incidental, regardless of the public causation dispute [44].

The institutional reversal mode is most cleanly illustrated by *Klarna*. In February 2024 *Klarna* issued a press release claiming its AI assistant had handled two-thirds of all customer service chats in its first month ("the equivalent work of 700 full-time agents"), with customer satisfaction scores equivalent to those of human agents and resolution time down from eleven minutes to two [53]. By May 2025, CEO Sebastian Siemiatkowski told Bloomberg that "as cost unfortunately seems to have been a too predominant evaluation factor when organizing this, what you end up having is lower quality" [94]; *Klarna* began rehiring human agents in a hybrid model [91]. *Klarna*'s substitution involved customer service rather than software development, but the structural lesson transfers: a public AI-replaces-humans strategy can require a public reversal, and the cost-of-undo (including the quality deterioration during the AI-only period and the reputational cost of an executive walking back a boosterist statement) is systematically under-priced in the vendor-favouring productivity analyses.

6.2 Code quality at scale

The vendor-reported evidence on code quality points consistently in the same direction. *CodeRabbit*'s December 2025 analysis of 470 open-source pull requests [63] found 1.7× more issues per pull request in AI co-authored code (10.83 against 6.45), with critical

findings up 1.4×, logic and correctness errors up 1.75×, and security findings up 1.57×; authorship was inferred from co-author annotations rather than confirmed directly, which the report flags as its principal limitation. *Veracode*'s October 2025 State of Software Security report [110] found that the security profile of AI-generated code is not improving despite measurable gains in functional correctness. *GitClear*'s longitudinal study [32], tracking 153 million lines of code changes from 2021 to 2024, found refactoring's share of all changes falling from 25% to below 10%, code duplication rising approximately fourfold, and churn nearly doubling, exactly the pattern one would expect from a workflow in which the cost of generating new code has fallen sharply while the cost of understanding existing code has not. *Faros AI*'s 2026 telemetry [27] shows the downstream consequence directly: across 22,000 developers, the task-completion gains of Table 3 were accompanied by review time, defect counts, and incidents per pull request all rising several times faster than the gains themselves. Each of these datasets is vendor-produced and should be read with the corresponding commercial-interest caveat; their convergence on the same direction of effect across four independent measurement systems is nonetheless informative.

The theoretical framing for these empirical patterns comes from Koren, Békés, Hinz and Lohmann's January 2026 equilibrium model [55], funded by an ERC Advanced Grant. The model treats the open-source software market as a coupled equilibrium with endogenous project entry, heterogeneous quality, and maintainer compensation flowing through user engagement (bug reports, documentation contributions, community recognition) rather than direct payment. Vibe coding raises productivity by lowering the cost of using and assembling open-source components (a positive demand shock) while simultaneously weakening the user engagement through which maintainers earn their non-monetary returns (a negative supply shock): the archetype vibe coder reads neither the generated code nor the upstream packages, and so does not file bug reports, open documentation issues, or contribute to the community recognition that compensates maintainers. The model's headline result is that overall welfare decreases despite individual productivity gains, because the gains are realised disproportionately by users who do not internalise the cost of the public good they consume. Linus Torvalds's "horrible idea for maintenance" assessment [106] and Daniel Stenberg's forced closure of *curl*'s bug-bounty programme [102] are the empirical instantiations of this theoretical prediction at the maintainer level.

6.3 Copyright and IP risk

The first concrete legal stress test arrived in March 2026, when Anthropic exposed approximately 512,000 lines of the *Claude Code* source in a mispackaged npm release [112] and AI-mediated reimplementations were published to GitHub within forty-eight hours, one of them reportedly reaching a hundred thousand stars in a single day [24]. Anthropic's DMCA campaign reached direct copies of the leaked code but not the AI-rewritten reimplementations, which persisted on decentralised platforms outside U.S. jurisdiction [56].

The episode exposed three unsettled questions that apply to vibe-coded codebases generally [56]. Clean-room doctrine requires that a reimplementing team have no access to the original; when the original is instead supplied as input to an AI system, no court has ruled on whether the output is "independent creation". Copyright protection requires meaningful human authorship, which a substantially AI-authored codebase may lack — the creator of *Claude Code* is on record that "100% of code is written by Claude Code, I haven't edited a single line since November" [54] — so a vendor in Anthropic's position may struggle to enforce copyright in its own product. And when a proprietary codebase can be rewritten overnight into a form beyond the reach of takedown mechanisms, large-scale exposure becomes effectively irreversible. The property that makes vibe coding attractive, reduced human contribution per line, is the same one that current doctrine treats as insufficient to anchor copyright; trade secrecy remains the principal unaffected lever, and only for code that is never exposed.

6.4 Skill atrophy and the labour market

The cognitive evidence on skill atrophy under AI-assisted workflows is now grounded in at least three empirical studies, two of which originate from AI vendors and reach conclusions adverse to their commercial interest. Lee, Sarkar, Tankelevitch and colleagues at *Microsoft Research*, in a CHI 2025 study of 319 knowledge workers [60], found that higher confidence in AI tools was associated with reduced critical thinking, while higher self-confidence in one's own skills was associated with maintained critical thinking, the "confidence mediation" effect. *Anthropic*'s January 2026 randomised controlled trial [93] assigned fifty-two mostly-junior software engineers to AI-assisted or hand-coding conditions on exercises in the Python library *Trio*; the AI-assisted group finished approximately two minutes faster (not statistically significant) but scored seventeen percentage points lower on a comprehension quiz administered immediately afterward, with the four heavy delegators averaging below 40% and the five participants who asked follow-up questions and requested explanations averaging above 65%, though those sub-groups are too small to carry weight on their own. *Anthropic*'s own framing acknowledges the trade-off: "productivity benefits may come at the cost of skills necessary to validate AI-written code if junior engineers' skill development has been stunted by using AI in the first place" [93]. A Frontiers in Psychology study of 1,032 university students found that AI dependence significantly increased cognitive inertia, which in turn significantly reduced innovation capability [119].

The labour-market consequences of the skill-development concern are visible in entry-level hiring data. Brynjolfsson, Chandar and Chen's *Stanford Digital Economy Lab* working paper [15], using *ADP* payroll data through September 2025, documents approximately a 20% decline in employment for 22–25-year-old software developers from its peak in late 2022 (contemporaneous with the launch of *ChatGPT*), with a broader 16% relative decline across the most AI-exposed occupations once firm-level shocks are controlled for. US computer-science enrolment in four-year programs fell 8.1% in the 2025–26 academic year, the steepest decline of any field of study [74], and the CS-specific decline was 11.2%. The coding-bootcamp sector has contracted sharply across the same period [116]. *Kenzie Academy* (operated by Southern New Hampshire University) closed in August 2023, with a university spokeswoman citing the "exponential" adoption of artificial intelligence as a factor in the decision [115]. *Momentum Learning* closed in April 2024, its

co-founder stating that "generative AI's influence on entry-level coding jobs" and AI tools now coding at novice level made the business unviable [90]. *Epicodus* saw enrolment fall by more than 75% year-over-year by its January 2024 cohort, though the organisation did not publicly attribute the decline to AI specifically [116]. Not every closure in the sector is attributable to AI: *2U*, the online programme manager behind edX, filed for Chapter 11 in July 2024 under the debt it had taken on to buy edX from Harvard and MIT for $800M in 2021, a collapse its coverage attributes to financial structure, competition and regulation rather than to AI [20]. The collateral consequence is structural and slow-moving: if organisations stop hiring junior developers today, they will have no senior developers in five to ten years; the apprenticeship pipeline by which senior developers are produced is being severed at precisely the moment when the demand for developers who can validate AI output is rising.

6.5 Mitigation strategies

The vendor-side answer to the comprehension gap is a family of learning-oriented modes: *Claude Code* ships built-in *Explanatory* and *Learning* output styles, the first interleaving educational "Insights" with the work and the second asking the developer to write small, strategic pieces of the implementation at `TODO(human)` markers [8], while *ChatGPT*'s study mode, introduced for learners in July 2025, replaces direct answers with step-by-step guiding questions [78]. The design intent matches the *Anthropic* trial's own finding that comprehension was retained by the participants who asked follow-up questions and requested explanations rather than delegating wholesale [93]. The chronology does not, however, support reading these modes as a response to that evidence: both shipped in mid-2025, months before the trial, which cites them as existing features rather than proposing them, and neither has been evaluated for whether it closes the gap it is invoked against.

The open-source instruments introduced in Section 2.3 are governance rather than tooling, and they encode the Willison boundary as a formal rule. The Linux kernel policy requires AI-assisted contributions to carry an `Assisted-by:` trailer and forbids AI agents from certifying developer identity through `Signed-off-by:`, so that AI may review but not sign as author [22]; the Rust project's ban on "vibecoded" contributions says the same thing more bluntly [86]. Stenberg's closure of *curl*'s bug-bounty programme is the opposite response, institutional self-defence by withdrawal rather than by policy [102].

Enterprise practice is arriving at the same shape from the other direction. *Amazon*'s 90-day "code safety reset", which routed changes to around 335 critical systems through two-person review and a formal documentation-and-approval step [44], is the most prominent documented example, and its logic follows the skill-atrophy evidence: the comprehension cost of AI-assisted coding is lowest where the human in the loop has enough expertise to verify the output.

6.6 The compound risk picture

Read together, the registers above describe five exposures that are structurally distinct from the individual productivity calculus and that no headline productivity figure prices. The *maintenance cliff* accrues invisibly: a vibe-coded codebase that passes its tests on day one ages into a system that nobody in the organisation can read, modify, or debug without AI assistance, which is the maintainer-level dynamic of Section 6.2 arriving inside the firm. The *skill-pipeline* exposure runs on a five-to-ten year horizon, since organisations that stop hiring and training juniors today face the senior-talent shortage of the mid-2030s (Section 6.4). *Incident exposure* is structural rather than exceptional: the failures of Section 6.1 share a single pattern, confidence without comprehension deployed into production, and the population scans suggest it is the modal outcome rather than the outlier. The *cost of undo* is what *Klarna* makes concrete, and it is not an isolated reversal: a *KPMG* survey of 2,145 executives across twenty countries found nearly half of organisations had "rephased" AI deployments whose costs outweighed the value delivered, even as headline confidence in AI held steady [57, 72]. The *accountability vacuum* is the most concrete enterprise-level finding in the corpus: in the *CloudBees* survey 93% of organisations report a formal review-and-release process for AI-generated code but only 56% say it is always enforced, and when that code causes a production failure accountability defaults upward, landing on the CTO or VP of Engineering in 46% of organisations and on the developer who shipped the change in 7% [19]. That is the exact inverse of the open-source response, which insists on per-patch human accountability (Section 6.5), and the contrast is the sharpest available statement of where the two ecosystems now differ.

7 Discussion and Conclusion

7.1 Where the trajectory points

Three readings of the assembled evidence extend past the window this review covers.

The first concerns the capability ceiling. Whether *SWE-Bench Verified* at 95% represents genuine saturation or a benchmark artefact is answered, as far as the corpus allows, by the gaps documented in Section 4.1: the same models lose fifteen to nineteen points on the contamination-resistant *Pro* set, and lose more again when the evaluation is run independently rather than self-reported. The residual few per cent on Verified is therefore better read as dataset noise than as a capability boundary, and the benchmarks that will discriminate between systems over the next two years are the ones with the most headroom left today. The field's measurement problem, on current evidence, will outlast its capability problem.

The second is the narrowing of the open/closed divide. The cost-adjusted picture of Section 3.3, a fourteen-percentage-point accuracy gap at a cost ratio above twenty to one, already favours open-weights self-hosting for the large majority of work that does not need the absolute frontier, and the accuracy gap has been closing faster than the price gap. If that continues, the binding constraint on who can vibe-code at scale becomes GPU capital rather than per-token spend, which distributes access differently than the free-tier era did.

The third is the bifurcation that the open-source community reached first. Stenberg's FOSDEM 2026 line, "AI gives us the worst and the best—simultaneously" [104], states in one sentence the distinction that the kernel's tagging policy, the Rust project's ban on vibecoded contributions, and Willison's definitional boundary

each encode in their own register [22, 86, 118]: AI as reviewer and analyser is accepted, AI as autonomous author is not. That this convergence happened within a twelve-month window, among maintainers with no particular reason to coordinate, is the clearest directional signal in the corpus, and Section 6.5 shows enterprise governance arriving at the same line from the opposite direction.

7.2 Open research questions

Four questions operate on longer cycles than any evidence assembled here can resolve. The most consequential is the long-term trajectory of developer comprehension under sustained AI assistance: the single-session gap measured by *Anthropic* [93] says nothing about whether comprehension recovers on return to unassisted work, compounds over years of practice, or settles at a lower equilibrium, and no study in the corpus can distinguish the three. The maintenance economics of vibe-coded codebases over three-to-five year horizons is equally unmeasured: *GitClear* documents the early signal [32], but the point at which accumulated debt makes such a codebase cheaper to replace than to maintain is not visible in any dataset. The legal status of AI-mediated clean-room reimplementation, crystallised by the *Claude Code* leak but not yet adjudicated, will shape the industry's IP economics once a court addresses the questions raised by Koustenis and Gregg [56].

The fourth question is methodological, and it bounds this review's own reading of the productivity record: no study in the corpus stratifies AI-assisted productivity by codebase age. The evidence is consistent with the conjecture that the short-horizon gains of the early trials would not survive re-measurement on mature code, since the largest displacement claims describe greenfield-skewed conditions while the one independent trial on mature repositories measured a slowdown [13, 39, 105], but no study was designed to test that axis. A randomised comparison of identical tasks across greenfield and five-to-ten-year-old codebases would corroborate the six patterns of Section 5.3 or falsify them; until one exists, those patterns remain a structurally consistent reading of the corpus rather than an isolated empirical finding.

7.3 Conclusion

The central tension of the field is the divergence of three curves that run on different time scales but move, over the period this review covers, in visibly different directions (Figure 4). Capability, as measured by benchmark scores, has moved from 1.7% to 95% on *SWE-Bench Verified* in roughly thirty months, an extraordinary improvement by any prior standard. Measured productivity for experienced developers on mature codebases moved the other way over the same period, from the vendor trials' +55% to *METR*'s −19%, and has recovered only marginally since [13, 69]. The population-level skill cost, visible in the junior comprehension gap, the entry-level hiring decline, and the fall in computer-science enrolment, is compounding on a five-to-ten year horizon that none of the productivity studies was built to measure. All three curves are real, which is why the public argument about vibe coding is so hard to settle: two well-informed participants can reach opposite conclusions by reading different curves, and both can be right about the curve they are reading.

The codebase-age conjecture stated in Section 1.2 is our best account of why: if the gains are real on new code and shrink or reverse on mature code, most of the dispersion in the record follows without any party to the argument having measured badly. What the corpus supports, then, is neither the displacement narrative nor its rejection. Vibe coding is a durable change in how software is written, whose benefits are narrower, more population-specific, and shorter-horizon than the headline figures claim, and whose costs are slower, more diffuse, and harder to attribute than the failure anecdotes suggest. Which of those dominates over the next five years depends on whether capability improves fast enough to compensate for the skill cost, whether the labour market settles at a new equilibrium, or whether a sequence of high-profile failures forces institutional correction. On present evidence the field is better equipped to make that question urgent than to answer it.

Acknowledgments

This research has been supported by the baseline funding of the KAUST Computational Sciences Group. The authors thank Roland Ruiters and Oliver Burghard for helpful discussions. AI assistance for writing support was used throughout all sections of this paper (*Claude Opus 4.8* by *Anthropic*). It was used for drafting text based on collected notes. All of these were subsequently reworked into the final paper text by the authors. AI was furthermore used to write the Python code for plotting the figures in this paper, to help with online research for additional sources, and to write artificial reviews of paper drafts to help identifying weaknesses.

References

[1] Leonel Acevedo. 2025. guys, i'm under attack. Post on X (Twitter), @leojr94_. https://archive.today/2025.09.15-153256/https://x.com/leojr94_/status/1901560276488511759

[2] Leonel Acevedo. 2025. i'm shutting down my app. Post on X (Twitter), @leojr94_. https://archive.today/2025.09.15-152520/https://x.com/leojr94_/status/1902537756674318347

[3] Leonel Acevedo. 2025. my saas was built with Cursor, zero hand written code. Post on X (Twitter), @leojr94_. https://archive.today/2025.09.15-154014/https://x.com/leojr94_/status/1900767509621674109

[4] Ammar Ahmed and Ali Shariq Imran. 2025. The role of large language models in UI/UX design: A systematic literature review. arXiv:2507.04469 [cs.HC] https://arxiv.org/abs/2507.04469

[5] AISLE. 2026. AISLE discovered 12 out of 12 OpenSSL vulnerabilities. AISLE company blog (aisle.com). https://web.archive.org/web/20260604030904/https://aisle.com/blog/aisle-discovered-12-out-of-12-openssl-vulnerabilities Vendor-published; the findings are independently corroborated by the coordinated OpenSSL release and CVE assignments.

[6] Nadia Alshahwan, Jubin Chheda, Anastasia Finegenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE 2024), Industry Track*. arXiv:2402.09171 [cs.SE] doi:10.1145/3663529.3663839

[7] Anthropic. 2025. How AI Is Transforming Work at Anthropic. Anthropic research blog. https://web.archive.org/web/20260512225351/https://www.anthropic.com/research/how-ai-is-transforming-work-at-anthropic

[8] Anthropic. 2026. Output styles. Claude Code documentation (code.claude.com). https://web.archive.org/web/20260717153245/https://code.claude.com/docs/en/output-styles.md

[9] Anthropic. 2026. Pricing. Claude platform documentation. https://web.archive.org/web/20260728010255/https://platform.claude.com/docs/en/about-claude/pricing

[10] Anthropic. 2026. Project Glasswing. Anthropic announcement. https://web.archive.org/web/20260725212953/https://anthropic.com/glasswing

[11] Anysphere. 2026. Cursor pricing. Cursor product page. https://web.archive.org/web/20260728002035/https://cursor.com/pricing

[12] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le,

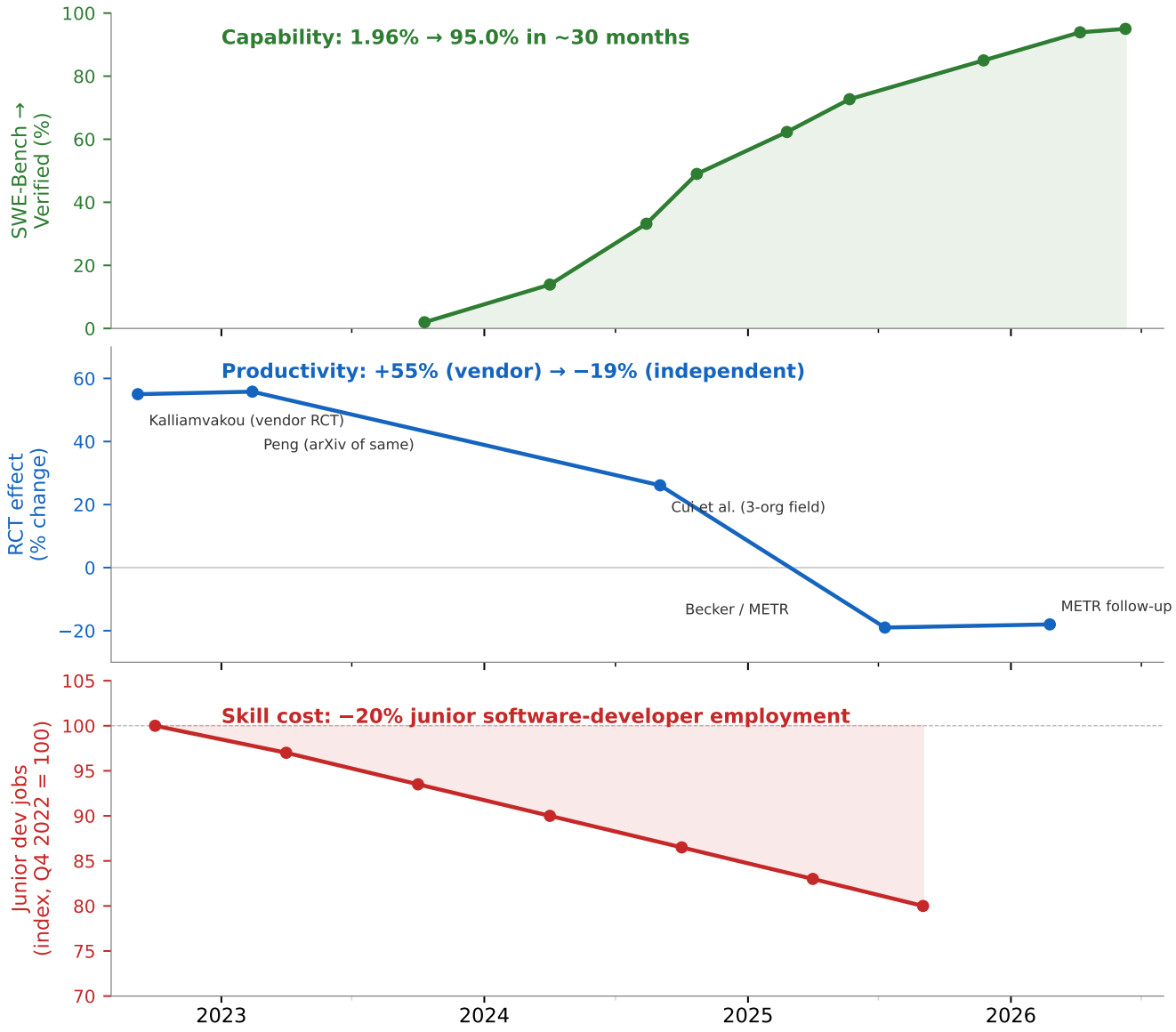


Figure 4: The three divergent curves of the field. *Top*: capability on *SWE-Bench* and its *Verified* successor, rising from 1.96% at the original benchmark's launch in October 2023 [46] to 95.0% in approximately thirty months [77, 108]. *Middle*: productivity in randomised studies shifting from +55.8% in the original 95-developer vendor RCT [81] through +26.1% in Cui et al.'s 4,867-developer field experiment [23] to −19% in *METR*'s independent RCT of sixteen experienced developers [13], with the follow-up cohort at −18% [69]. *Bottom*: junior software-developer employment indexed to its late-2022 peak, −20% by September 2025 [15]. Intermediate capability points are interpolated from public leaderboard milestones; intermediate skill-cost points are linearly interpolated from the Brynjolfsson et al. narrative endpoint.

and Charles Sutton. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732 [cs.PL] https://arxiv.org/abs/2108.07732

[13] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. arXiv:2507.09089 [cs.SE] https://arxiv.org/abs/2507.09089

[14] Arnaud Benard. 2025. [Post: Galileo AI acquired by Google; rebranded as Google Stitch]. Post on X (Twitter), @arnaudai. https://archive.today/2026.07.19-132306/https://x.com/arnaudai/status/1924942577545195982

[15] Erik Brynjolfsson, Bharat Chandar, and Ruyu Chen. 2025. Canaries in the Coal Mine? Six Facts about the Recent Employment Effects of Artificial Intelligence. Stanford Digital Economy Lab working paper. https://web.archive.org/web/20260109130745/https://digitaleconomy.stanford.edu/publications/canaries-in-the-coal-mine/

[16] Yihan Cao, Siyu Li, Yixin Liu, Zhiling Yan, Yutong Dai, Philip S. Yu, and Lichao Sun. 2023. A Comprehensive Survey of AI-Generated Content (AIGC): A History of Generative AI from GAN to ChatGPT. arXiv:2303.04226 [cs.AI] https://arxiv.org/abs/2303.04226

[17] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] https://arxiv.org/abs/2107.03374

[18] Bei Chu, Yang Feng, Kui Liu, Zhaoqiang Guo, Yichi Zhang, Hange Shi, Zifan Nan, and Baowen Xu. 2025. Large Language Models for Unit Test Generation: Achievements, Challenges, and Opportunities. arXiv:2511.21382 [cs.SE] https://arxiv.org/abs/2511.21382

[19] CloudBees. 2026. The State of Code Abundance 2026: The Gap Between AI Output and Enterprise Control. CloudBees vendor report. https://web.archive.org/web/20260524224859/https://www.cloudbees.com/blog/2026-state-of-code-abundance-report

[20] Lauren Coffey. 2024. Long-embattled 2U declares bankruptcy. *Inside Higher Ed* (26 July 2024). https://web.archive.org/web/20251008125856/https://www.insidehighered.com/news/business/2024/07/26/long-embattled-2u-declares-bankruptcy

[21] Collins English Dictionary. 2025. Word of the Year 2025: vibe coding. https://web.archive.org/web/20260519063021/https://www.collinsdictionary.com/woty

[22] Jonathan Corbet. 2025. Toward a policy for machine-learning tools in kernel development. Linux Weekly News (LWN), article 1049830. https://web.archive.org/web/20260216160942/https://lwn.net/Articles/1049830/

[23] Zheyuan (Kevin) Cui, Mert Demirer, Sonia Jaffe, Leon Musolff, Sida Peng, and Tobias Salz. 2024. The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers. *Management Science* (2024). doi:10.1287/mnsc.2025.00535

[24] Cybernews. 2026. Leaked Claude Code source spawns fastest growing repository in GitHub's history. Cybernews. https://web.archive.org/web/20260412012336/https://cybernews.com/tech/claude-code-leak-spawns-fastest-github-repo/

[25] Khalid El Haji, Carolin Brandt, and Andy Zaidman. 2024. Using GitHub Copilot for Test Generation in Python: An Empirical Study. In *5th ACM/IEEE International Conference on Automation of Software Test (AST 2024), co-located with ICSE*. 45–55. doi:10.1145/3644032.3644443

[26] Escape.tech. 2025. The State of Security of Vibe Coded Apps. Escape.tech research report. https://web.archive.org/web/20260714131114/https://escape.tech/state-of-security-of-vibe-coded-apps

[27] Faros AI. 2026. Key takeaways from the 2025 DORA report (Faros 2026 telemetry update). Faros AI industry report. https://web.archive.org/web/20260514032238/https://www.faros.ai/blog/key-takeaways-from-the-dora-report-2025

[28] Ahmed Fawzy, Amjed Tahir, and Kelly Blincoe. 2025. Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook — a Grey Literature Review. arXiv:2510.00328 [cs.SE] https://arxiv.org/abs/2510.00328

[29] Lorenzo Franceschi-Bicchierai. 2025. Google says its AI-based bug hunter found 20 security vulnerabilities. TechCrunch. https://web.archive.org/web/20260402181704/https://techcrunch.com/2025/08/04/google-says-its-ai-based-bug-hunter-found-20-security-vulnerabilities/

[30] Paul Gauthier. 2025. Aider LLM Leaderboards: Polyglot. Aider (open-source project). https://web.archive.org/web/20260520122745/https://aider.chat/docs/leaderboards/

[31] Yuyao Ge, Lingrui Mei, Zenghao Duan, Tianhao Li, Yujia Zheng, Yiwei Wang, Lexin Wang, Jiayu Yao, Tianyu Liu, Yujun Cai, Baolong Bi, Fangda Guo, Jiafeng Guo, Shenghua Liu, and Xueqi Cheng. 2025. A Survey of Vibe Coding with Large Language Models. arXiv:2510.12399 [cs.AI] https://arxiv.org/abs/2510.12399

[32] GitClear. 2025. AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones. GitClear industry report. https://web.archive.org/web/20260510013635/https://www.gitclear.com/ai_assistant_code_quality_2025_research

[33] GitHub. 2025. Octoverse 2025: The state of open source. GitHub Octoverse annual report. https://web.archive.org/web/20260519041111/https://octoverse.github.com/

[34] GitHub. 2026. GitHub Copilot plans. GitHub product documentation. https://web.archive.org/web/20260727153657/https://github.com/features/copilot/plans

[35] Emily Glassberg Sands. 2025. Agentic Commerce Protocol and building the Economic Infrastructure for AI. Interview, Latent Space podcast. https://web.archive.org/web/20260416212228/https://www.latent.space/p/stripe

[36] Google Cloud / DORA. 2024. Accelerate State of DevOps 2024: The Impact of AI on Software Delivery. DORA / Google Cloud industry report. https://web.archive.org/web/20260409231915/https://dora.dev/research/2024/dora-report/

[37] Google Cloud / DORA. 2025. State of AI-assisted Software Development 2025. DORA / Google Cloud industry report. https://web.archive.org/web/20250923175604/https://services.google.com/fh/files/misc/2025_state_of_ai_assisted_software_development.pdf

[38] Google Project Zero. 2024. From Naptime to Big Sleep: using large language models to catch vulnerabilities in real-world code. Google Project Zero blog. https://web.archive.org/web/20260722031007/https://projectzero.google/2024/10/from-naptime-to-big-sleep.html

[39] Alistair Gray. 2026. Minions: Stripe's one-shot, end-to-end coding agents—Part 2. Stripe Developer Blog (stripe.dev). https://archive.today/2026.02.20-174514/https://stripe.dev/blog/minions-stripes-one-shot-end-to-end-coding-agents-part-2

[40] Ian Guelman, Arthur Gregório Leal, Laerte Xavier, and Marco Tulio Valente. 2024. On the Quality of AI-Generated Source Code Comments: A Comprehensive Evaluation. arXiv:2408.14007 [cs.SE] https://arxiv.org/abs/2408.14007

[41] Martin Gundtoft. 2025. Vibe Coding a Research Probe for Exploring AI/Voice Based Code Reviews. In *Proceedings of the International Conference on AI Research (ICAIR 2025)*, Vol. 6. Academic Conferences International, Business Academy Copenhagen, 672–675. doi:10.34190/icair.5.1.3975

[42] Infosecurity Magazine. 2026. Autonomous system uncovers long-standing OpenSSL flaws. Infosecurity Magazine. https://web.archive.org/web/20260219004446/https://www.infosecurity-magazine.com/news/12-openssl-flaws/

[43] Jacob Jackson. 2018. TabNine's first month in review. Personal blog (jacobjackson.com). https://web.archive.org/web/20260301081305/https://jacobjackson.com/first-month/

[44] Luke Jacoby. 2026. AI code wreaked havoc with Amazon outage, and now the company is making tight rules. Digital Trends. https://web.archive.org/web/20260515042245/https://www.digitaltrends.com/computing/ai-code-wreaked-havoc-with-amazon-outage-and-now-the-company-is-making-tight-rules/

[45] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. arXiv:2403.07974 [cs.SE] https://arxiv.org/abs/2403.07974

[46] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770 [cs.CL] https://arxiv.org/abs/2310.06770

[47] Eirini Kalliamvakou. 2022. Research: quantifying GitHub Copilot's impact on developer productivity and happiness. GitHub blog. https://web.archive.org/web/20260519040538/https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/

[48] Sungmin Kang, Louis Milliken, and Shin Yoo. 2024. Identifying Inaccurate Descriptions in LLM-generated Code Comments via Test Execution. arXiv:2406.14836 [cs.SE] https://arxiv.org/abs/2406.14836

[49] Begum Karaci Deniz, Chandra Gnanasambandam, Martin Harrysson, Alharith Hussin, and Shivam Srivastava. 2023. Unleashing developer productivity with generative AI. McKinsey & Company explainer. https://web.archive.org/web/20260516081336/https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/unleashing-developer-productivity-with-generative-ai

[50] Andrej Karpathy. 2025. There's a new kind of coding I call "vibe coding". Post on X (Twitter). https://archive.today/2026.05.07-152819/https://x.com/karpathy/status/1886192184808149383

[51] Andrej Karpathy. 2026. One-year retrospective on the original "vibe coding" tweet. Post on X (Twitter). https://archive.today/2026.02.05-000704/https://x.com/karpathy/status/2019137879310836075

[52] Malte Kirchner. 2026. Analyse zur Google I/O: Überfällige Ehrlichkeit. heise online (Meinung). https://web.archive.org/web/20260520144130/https://www.heise.de/meinung/Analyse-zur-Google-I-O-Ueberfaellige-Ehrlichkeit-11299955.html

[53] Klarna. 2024. Klarna AI assistant handles two-thirds of customer service chats in its first month. Klarna press release. https://web.archive.org/web/20260513221710/https://www.klarna.com/international/press/klarna-ai-assistant-handles-two-thirds-of-customer-service-chats-in-its-first-month/

[54] Andrey Kolkov. 2026. We Reverse-Engineered 12 Versions of Claude Code. Then It Leaked Its Own Source Code. DEV Community (dev.to). https://web.archive.org/web/20260401081616/https://dev.to/kolkov/we-reverse-engineered-12-versions-of-claude-code-then-it-leaked-its-own-source-code-pij

[55] Miklós Koren, Gábor Békés, Julian Hinz, and Aaron Lohmann. 2026. Vibe Coding Kills Open Source. arXiv:2601.15494 [econ.GN] https://arxiv.org/abs/2601.15494

[56] Kandis Koustenis and Andrew Gregg. 2026. 512,000 Lines, One Night, Zero Permission: The Claude Code Leak and the Legal Crisis of AI Clean Rooms. Bean, Kinney & Korman P.C. (legal analysis). https://web.archive.org/web/20260518143454/https://www.beankinney.com/512000-lines-one-night-zero-permission-the-claude-code-leak-and-the-legal-crisis-of-ai-clean-rooms/

[57] KPMG. 2026. Global AI Pulse: Q2 2026. KPMG survey report; 2,145 C-suite leaders across 20 countries. https://web.archive.org/web/20260714121551/https://kpmg.com/xx/en/our-insights/ai-and-technology/ai-pulse.html

[58] Frederic Lardinois. 2019. Kite raises $17M for its AI-driven code completion tool. TechCrunch. https://web.archive.org/web/20190128205439/https://techcrunch.com/2019/01/28/kite-raises-17m-for-its-ai-driven-code-completion-tool/

[59] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *Nature* 521, 7553 (May 2015), 436–444. doi:10.1038/nature14539

[60] Hao-Ping (Hank) Lee, Advait Sarkar, Lev Tankelevitch, Ian Drosos, Sean Rintel, Richard Banks, and Nicholas Wilson. 2025. The Impact of Generative AI on Critical Thinking: Self-Reported Reductions in Cognitive Effort and Confidence Effects From a Survey of Knowledge Workers. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI 2025)*. doi:10.1145/3706598.3713778

[61] LLM-Stats. 2026. SWE-bench Pro Leaderboard. llm-stats.com; living leaderboard. https://web.archive.org/web/20260724211801/https://llm-stats.com/benchmarks/swe-bench-pro All listed entries are vendor self-reported; none independently verified as of the urldate..

[62] LLM-Stats. 2026. SWE-bench Verified Leaderboard. llm-stats.com; aggregated reported scores (living leaderboard). https://web.archive.org/web/20260723022001/https://llm-stats.com/benchmarks/swe-bench-verified

[63] Dan Loker. 2025. State of the AI vs. Human Code Generation Report. CodeRabbit whitepaper (vendor report). https://web.archive.org/web/20260517080519/https://www.coderabbit.ai/whitepapers/state-of-AI-vs-human-code-generation-report

[64] Tobias Lütke. 2025. Reflexive AI usage is now a baseline expectation at Shopify. Post on X (Twitter), @tobi. https://archive.today/2025.07.12-011528/https://x.com/tobi/status/1909251946235437514

[65] Sanghee Ma and Dongyeop Kang. 2026. SWE-ABS: Adversarial Benchmark Strengthening for SWE-bench. arXiv:2603.00520 [cs.SE] https://arxiv.org/abs/2603.00520

[66] Chris McKay. 2024. Galileo AI Raises $4.4M and Launches Public Beta of Prompt-to-UI AI Design Tool. Maginative. https://web.archive.org/web/20260211152453/https://www.maginative.com/article/galileo-ai-raises-launches-public-beta-of-text-to-ui-generative-design-tool/

[67] Merriam-Webster. 2025. Slang & Trending: vibe coding. Merriam-Webster dictionary. https://web.archive.org/web/20260429160023/https://www.merriam-webster.com/slang/vibe-coding

[68] Christian Meske, Tobias Hermanns, Esther von der Weiden, Kai-Uwe Loser, and Thorsten Berger. 2025. Vibe Coding as a Reconfiguration of Intent Mediation in Software Development: Definition, Implications, and Research Agenda. *IEEE Access* 13 (2025), 213242–213259. arXiv:2507.21928 https://arxiv.org/abs/2507.21928

[69] METR. 2026. We are Changing our Developer Productivity Experiment Design. METR blog. https://web.archive.org/web/20260518113231/https://metr.org/blog/2026-02-24-uplift-update/

[70] Microsoft and co-signatories. 2026. Open Weights and American AI Leadership. Microsoft corporate-responsibility statement; industry position paper co-signed by approximately fifty organisations (incl. Google, Meta, OpenAI, NVIDIA, Mistral, IBM, Hugging Face, GitHub, Replit). https://web.archive.org/web/20260726082911/https://www.microsoft.com/en-us/corporate-responsibility/topics/open-weight/ Industry advocacy; commercial interest. Anthropic is not among the signatories..

[71] MIT Technology Review. 2026. 10 Breakthrough Technologies 2026. MIT Technology Review. https://web.archive.org/web/20260521000057/https://www.technologyreview.com/2026/01/12/1130697/10-breakthrough-technologies-2026/

[72] Mike Moore. 2026. CEOs are being left baffled at the high cost of moving to AI. TechRadar Pro. https://web.archive.org/web/20260710111629/https://www.techradar.com/pro/ceos-are-being-left-baffled-at-the-high-cost-of-moving-to-ai-shockingly-enough-sacking-human-workers-isnt-resulting-in-huge-savings

[73] Gal Nagli. 2026. Hacking Moltbook: AI Social Network Reveals 1.5M API Keys. Wiz Blog. https://web.archive.org/web/20260714130527/https://www.wiz.io/blog/exposed-moltbook-database-reveals-millions-of-api-keys

[74] National Student Clearinghouse Research Center. 2026. Current Term Enrollment Estimates: Fall 2025. National Student Clearinghouse Research Center. https://web.archive.org/web/20260515170120/https://nscresearchcenter.org/current-term-enrollment-estimates/

[75] Peter Ng, Rohit Chouhan, and Tom Duncalf. 2025. Introducing Figma Make: A New Way to Test, Edit, and Prompt Designs. Figma Blog. https://web.archive.org/web/20260331005510/https://www.figma.com/blog/introducing-figma-make/

[76] Jordan Novet and Jonathan Vanian. 2025. Satya Nadella says as much as 30% of Microsoft code is written by AI. CNBC. https://web.archive.org/web/20260108222656/https://www.cnbc.com/2025/04/29/satya-nadella-says-as-much-as-30percent-of-microsoft-code-is-written-by-ai.html

[77] OpenAI. 2024. Introducing SWE-bench Verified. OpenAI blog. https://web.archive.org/web/20260722163343/https://openai.com/index/introducing-swe-bench-verified/

[78] OpenAI. 2025. Introducing study mode. OpenAI blog. https://web.archive.org/web/20260723154017/https://openai.com/index/chatgpt-study-mode/

[79] OpenAI. 2026. Codex Security: now in research preview. OpenAI blog. https://web.archive.org/web/20260716122442/https://openai.com/index/codex-security-now-in-research-preview/ Productised from the internal Aardvark tool announced in private beta, October 2025. Beta-period counts are vendor self-reported.

[80] OpenAI. 2026. Why SWE-bench Verified no longer measures frontier coding capabilities. OpenAI blog. https://web.archive.org/web/20260723124436/https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/

[81] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590 [cs.SE] https://arxiv.org/abs/2302.06590

[82] Sundar Pichai. 2024. Alphabet Third Quarter 2024 Earnings Call. Alphabet Investor Relations, earnings call transcript. https://web.archive.org/web/20260516121603/https://s206.q4cdn.com/479360582/files/doc_financials/2024/q3/2024-q3-earnings-transcript.pdf

[83] Sundar Pichai. 2026. News from Google Cloud Next 2026. Google Blog. https://web.archive.org/web/20260428155954/https://blog.google/innovation-and-ai/infrastructure-and-cloud/google-cloud/cloud-next-2026-sundar-pichai/

[84] Partha Pratim Ray. 2025. A Review on Vibe Coding: Fundamentals, State-of-the-art, Challenges and Future Directions. TechRxiv preprint. doi:10.36227/techrxiv.174681482.27435614/v1

[85] Stuart J. Russell and Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach* (4 ed.). Pearson Education.

[86] Rust Project Leadership Council. 2026. Policy: Vibecoded contributions are not acceptable on rust-lang/rust. rust-lang/leadership-council, GitHub issue #273. https://web.archive.org/web/20260519043723/https://github.com/rust-lang/leadership-council/issues/273

[87] Sam Sabin. 2026. AI coding tools exposed sensitive corporate and personal data. *Axios* (7 May 2026). https://web.archive.org/web/20260707095007/https://www.axios.com/2026/05/07/loveable-replit-vibe-coding-privacy

[88] Advait Sarkar and Ian Drosos. 2025. Vibe coding: programming through conversation with artificial intelligence. In *Proceedings of the 36th Annual Conference of the Psychology of Programming Interest Group (PPIG 2025)*. arXiv:2506.23253 [cs.HC] https://arxiv.org/abs/2506.23253

[89] Scale AI. 2026. SWE-bench Pro Leaderboard (public set). Scale AI / labs.scale.com; independently run, standardised public set. https://web.archive.org/web/20260720130629/https://labs.scale.com/leaderboard/swe_bench_pro_public

[90] Astrid Schiavenzky. 2024. Triangle coding bootcamp closes, citing AI's influence on entry-level jobs. Axios Raleigh. https://web.archive.org/web/20260412053315/https://www.axios.com/local/raleigh/2024/04/01/triangle-coding-bootcamp-momentum-closes-citing-ai

[91] Michael Segalov. 2025. Klarna plans to hire humans again, as new landmark survey reveals most AI projects fail to deliver. Fortune. https://web.archive.org/web/20260511134247/https://fortune.com/2025/05/09/klarna-ai-humans-return-on-investment/

[92] Simon Sharwood. 2025. Vibe coding service Replit deleted user's production database, faked data, told fibs galore. The Register. https://web.archive.org/web/20260504195641/https://www.theregister.com/2025/07/21/replit_saastr_vibe_coding_incident/

[93] Justine H. Shen and Alex Tamkin. 2026. How AI Assistance Impacts the Formation of Coding Skills. Anthropic Research. https://web.archive.org/web/20260521083705/https://www.anthropic.com/research/AI-assistance-coding-skills

[94] Sebastian Siemiatkowski. 2025. Klarna Turns From AI to Real Person Customer Service. Bloomberg. https://archive.today/2026.06.11-024622/https://www.bloomberg.com/news/articles/2025-05-08/klarna-turns-from-ai-to-real-person-customer-service

[95] Jim Springfield. 2007. IntelliSense History, Part 1. Microsoft C++ Team Blog. https://web.archive.org/web/20260208232540/https://devblogs.microsoft.com/cppblog/intellisense-history-part-1/

[96] Stack Overflow. 2025. 2025 Developer Survey: AI. Stack Overflow industry survey. https://web.archive.org/web/20260402093804/https://stackoverflow.blog/2025/12/29/developers-remain-willing-but-reluctant-to-use-ai-the-2025-developer-survey-results-are-here/

[97] Peter Steinberger. 2025. Shipping at Inference-Speed. Personal blog (steipete.me). https://web.archive.org/web/20260505134542/https://steipete.me/posts/2025/shipping-at-inference-speed

[98] Peter Steinberger. 2026. The latest CodexBar update renders API costs wayyyy nicer [screenshot of $1,305,088.81 30-day OpenAI API bill]. Post on X (Twitter), @steipete. https://archive.today/2026.06.24-092943/https://x.com/steipete/status/2055346265869721905

[99] Peter Steinberger. 2026. OpenClaw, OpenAI and the future. Personal blog (steipete.me). https://web.archive.org/web/20260507092144/https://steipete.me/posts/2026/openclaw

[100] Peter Steinberger. 2026. People freaking out over my AI spend — what nobody sees [thread defending the OpenClaw workflow of ~100 continuous Codex agents]. Post on X (Twitter), @steipete. https://archive.today/2026.07.19-112458/https://x.com/steipete/status/2055405041843052792

[101] Daniel Stenberg. 2024. The I in LLM stands for intelligence. Personal blog (daniel.haxx.se). https://web.archive.org/web/20240102130931/https://daniel.haxx.se/blog/2024/01/02/the-i-in-llm-stands-for-intelligence/

[102] Daniel Stenberg. 2025. Death by a thousand slops. Personal blog (daniel.haxx.se). https://web.archive.org/web/20260519225256/https://daniel.haxx.se/blog/2025/07/14/death-by-a-thousand-slops/

[103] Daniel Stenberg. 2026. Mythos finds a curl vulnerability. Personal blog (daniel.haxx.se). https://web.archive.org/web/20260725205741/https://daniel.haxx.se/blog/2026/05/11/mythos-finds-a-curl-vulnerability/

[104] Daniel Stenberg. 2026. Open Source security in spite of AI. Personal blog (daniel.haxx.se); also FOSDEM 2026 closing keynote. https://web.archive.org/web/20260513023932/https://daniel.haxx.se/blog/2026/02/03/open-source-security-in-spite-of-ai/

[105] Garry Tan. 2025. For 25% of the Winter 2025 batch, 95% of lines of code are LLM generated. That's not a typo. The age of vibe coding is here. Post on X (Twitter), @garrytan. https://archive.today/2026.07.19-115400/https://x.com/garrytan/status/1897303270311489931

[106] The Register. 2025. Linus Torvalds Says Vibe Coding is Fine For Getting Started, 'Horrible Idea' For Maintenance. https://web.archive.org/web/20260502144454/https://www.theregister.com/2025/11/18/linus_torvalds_vibe_coding/

[107] The Register. 2026. Amazon insists AI coding isn't source of outages. The Register. https://web.archive.org/web/20260318102436/https://www.theregister.com/2026/03/10/amazon_ai_coding_outages/

[108] Vals AI. 2026. SWE-bench Verified. vals.ai; independently run evaluation (living leaderboard). https://web.archive.org/web/20260806004017/https://www.vals.ai/benchmarks/swebench

[109] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. 5998–6008. arXiv:1706.03762 [cs.CL] https://arxiv.org/abs/1706.03762

[110] Veracode. 2025. 2025 GenAI Code Security Report. Veracode industry report. https://web.archive.org/web/20260508223921/https://www.veracode.com/resources/analyst-reports/2025-genai-code-security-report/

[111] Vercel. 2023. Announcing v0: Generative UI. Vercel Blog. https://web.archive.org/web/20260413040621/https://vercel.com/blog/announcing-v0-generative-ui

[112] Brandon Vigliarolo. 2026. Anthropic goes nude, exposes Claude Code source by accident. The Register. https://web.archive.org/web/20260504133502/https://www.theregister.com/2026/03/31/anthropic_claude_code_source_code/

[113] Brandon Vigliarolo. 2026. Anthropic's bug-hunting Mythos was greatest marketing stunt ever, says cURL creator. The Register. https://web.archive.org/web/20260712173305/https://www.theregister.com/security/2026/05/11/anthropics-bug-hunting-mythos-was-greatest-marketing-stunt-ever-says-curl-creator/5238111

[114] Yao Wan, Zhangqian Bi, Yang He, Jianguo Zhang, Hongyu Zhang, Yulei Sui, Guandong Xu, Hai Jin, and Philip S. Yu. 2024. Deep Learning for Code Intelligence: Survey, Benchmark and Toolkit. *Comput. Surveys* 56, 12, Article 309 (Oct. 2024), 41 pages. doi:10.1145/3664597

[115] Emma Whitford. 2023. Southern New Hampshire U. to Close Coding Boot Camp. Inside Higher Ed. https://web.archive.org/web/20260423221757/https://www.insidehighered.com/news/tech-innovation/teaching-learning/2023/08/24/southern-new-hampshire-closing-coding-boot-camp

[116] Emma Whitford. 2025. Changes at Boot Camps Mark Signal Shifts in Workforce. Inside Higher Ed. https://web.archive.org/web/20260423221745/https://www.insidehighered.com/news/tech-innovation/teaching-learning/2025/01/09/changes-boot-camp-marks-signal-shifts-workforce

[117] Kyle Wiggers. 2022. With Kite's demise, can generative AI for code succeed? TechCrunch. https://web.archive.org/web/20250520221809/https://techcrunch.com/2022/12/10/with-kites-demise-can-generative-ai-for-code-succeed/

[118] Simon Willison. 2025. Not all AI-assisted programming is vibe coding. simonwillison.net. https://web.archive.org/web/20260518125105/https://simonwillison.net/2025/Mar/19/vibe-coding/

[119] Zhixin Yang, Hao Deng, and Nan Jiang. 2025. The impact mechanism of artificial intelligence dependence on college students' innovation capability: an empirical study from China. *Frontiers in Psychology* 16 (Dec. 2025), 1732837. doi:10.3389/fpsyg.2025.1732837

[120] Mingyue Yuan, Jieshan Chen, Yongquan Hu, Sidong Feng, Mulong Xie, Gelareh Mohammadi, Zhenchang Xing, and Aaron Quigley. 2024. Towards Human–AI Synergy in UI Design: Supporting Iterative Generation with LLMs. *ACM Transactions on Computer-Human Interaction* (2024). arXiv:2412.20071 [cs.HC] doi:10.1145/3773035

[121] Maxwell Zeff. 2025. Microsoft CEO says up to 30% of the company's code was written by AI. TechCrunch. https://web.archive.org/web/20260723024029/https://techcrunch.com/2025/04/29/microsoft-ceo-says-up-to-30-of-the-companys-code-was-written-by-ai/

[122] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2023. A Survey of Large Language Models. arXiv:2303.18223 [cs.CL] https://arxiv.org/abs/2303.18223

[123] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, et al. 2024. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. arXiv:2406.15877 [cs.SE] https://arxiv.org/abs/2406.15877